Go for JavaScript Developers

From React and Next.js to Go, and building real applications with Grit

Muke JohnBaptist

Written against Grit v3.403.0 and Go 1.25

Contents

Why Go, when you already ship JavaScript 2
What actually pushed you here 2
The shape of Go, in one page 3
What you give up, stated plainly 3
The one example that explains everything 4
What this book does 5
How to read it 6
Installing Go and your first program 8
Installing 8
Your first program 8
Capital letters are the access modifier 9
Modules: go.mod is your package.json 10
The commands you will actually use 11
Compiling a real binary 11
Variables, types, and the compiler that argues with you 14
Declaring things 14
The types you will actually use 15
Zero values: there is no undefined 16
No truthiness 16
Strings are bytes, and that matters 17
Control flow, briefly 18
Functions, multiple returns, and the end of try/catch 20
Functions 20
Two return values, and why 21
`error` is just an interface 21
Wrapping: adding context without losing the original 22
The argument about verbosity 23
Named returns and `defer` 24
Structs and methods, against objects and classes 28
Declaring and creating 28
Methods: functions with a receiver 29
Value receivers and pointer receivers 29
Composition instead of inheritance 30
Struct tags: metadata for encoders 31
Constructors are just functions 31

Comparing structs 32
Slices: the array that is not an array 36
Arrays have a fixed length, in the type 36
Slices are a view onto an array 36
append, and the thing it does not promise 37
Iterating 38
Passing slices to functions 39
Sorting 39
Maps, and why iteration order is random on purpose 42
Declaring and using 42
Missing keys return the zero value 43
Iteration order is deliberately random 43
Maps are reference types 44
Structs as values, and the thing you cannot do 44
Sets 45
Maps are not safe for concurrent use 45
Pointers, without the fear 48
Everything in Go is passed by value 48
Two operators 48
nil, and the panic you will meet 49
When to use a pointer 50
When not to use a pointer 50
The loop variable, and a trap that Go fixed 51
Pointers to struct fields 52
Packages, modules and imports, against npm and ESM 56
A package is a directory 56
Importing 57
The import path is the module path plus the directory 57
internal/ is a real access rule 58
Naming, and the stutter rule 58
Modules, versions and go.mod 59
Major versions are in the path 60
init, and why to avoid it 60
Interfaces, and why Go's point the other way 64
The mechanical part 64
The direction: interfaces belong to the consumer 65
Keep them small 65
The empty interface, and `any` 66
The nil interface trap 67

Interfaces you will meet constantly 68
Why this matters for testing 68
Designing with errors: what a package should return 72
Three kinds of error, and when to use each 72
Wrap, but not at every level 73
Error strings are a contract, lowercase and unpunctuated 74
Do not return both a value and an error 74
Handle an error once 75
Mapping errors at the boundary 75
panic is not an error 76
defer, panic and recover 80
defer: cleanup that cannot be forgotten 80
The three rules of defer 80
defer can change the return value 82
defer and errors from Close 82
panic: stop, this should not have happened 83
recover: only for a boundary 83
Goroutines, against the event loop 88
The one-keyword version 88
What is actually different 88
Waiting properly: sync.WaitGroup 89
Collecting results 90
Races, and the detector that finds them 91
Bounding how many run at once 92
When not to use a goroutine 93
Channels, and the select statement 98
A channel is a typed pipe 98
Unbuffered channels are a handshake 99
Buffered channels have room 99
Closing, and ranging 99
select: waiting on several channels 100
Timeouts, the most common select 101
Signalling with a done channel 101
The nil channel, and why it is useful 102
Worker pools 103
sync: mutexes, and when a channel is the wrong tool 108
Mutex: one goroutine at a time 108
Embedding the mutex 109
RWMutex: many readers, one writer 109

sync.Once: exactly once, no matter who asks 110
atomic: for single values 110
sync.Map, and why you probably do not want it 111
When to use which 111
Context: cancellation, deadlines, and request scope 116
The problem it solves 116
Creating one 116
Using one 117
Cancellation is cooperative 118
The standard library already honours it 118
Timeouts compose 118
Values, and why to be sparing 119
The conventions 119

0

Why Go, when you already ship JavaScript

You already build and ship software. You know React, you know Next.js, and you can take an idea to production without asking anybody how. So the first question this book owes you is not "what is Go", it is "why would you spend a month learning a second language when the one you have works".

Here is the honest answer, and then the honest list of what you give up.

What actually pushed you here

Most JavaScript developers do not come to Go out of curiosity. They come because something in their stack got painful in a specific way. Usually one of these four.

The deployment got heavy. Your Next.js app needs Node on the server, a `node_modules` directory with a few hundred megabytes in it, and a build step that has to run somewhere. You want to copy one file onto a server and run it. A compiled Go program is one file. It has no runtime to install, no package directory, and no build step on the target machine. You build it on your laptop and the output is the thing that runs.

The work stopped being request-and-respond. An API that answers a question in forty milliseconds is a perfect job for Node. A process that holds ten thousand open connections, or runs a job queue, or chews through a large file while staying responsive, is where the single-threaded event loop stops being an elegance and starts being a constraint you work around.

You wanted the errors earlier. In JavaScript, a renamed field is found by a user. TypeScript helps enormously and you should keep using it, but it is a layer on top of a language that does not have types, and it disappears at runtime. In Go the types are the language. A program that compiles has already had a whole class of mistake removed from it.

Somebody said the word "backend" and meant it. Payment handling, a scheduler, an audit trail, something that must still be correct at 3am when nobody is watching. Go was built by people who operate systems like that for a living, and it shows in what the language refuses to let you do.

> **IF YOU KNOW JAVASCRIPT**
> None of this makes Go better than JavaScript. It makes it different in ways that matter for one kind of work. You are not replacing React. In the second half of this book you will have a Go backend and a React frontend in the same project, talking to each other with types generated from one definition. That is the shape most people actually want.

The shape of Go, in one page

Before any syntax, here is the character of the language, because it explains every decision you are about to meet.

Go is **small on purpose**. The entire specification is about one hundred pages. There are twenty five keywords. JavaScript has roughly double that and a standard library that has grown for thirty years. You can hold all of Go in your head, which is a design goal rather than an accident, and it is why Go teams onboard people in days rather than weeks.

Go is **explicit on purpose**. There is no implicit type coercion, no `this` that changes meaning depending on how a function was called, no prototype chain to walk. When you read a line of Go, what happens is what is written. This is sometimes tedious and it is never surprising, and after a few weeks you will notice you have stopped debugging a category of problem entirely.

Go is **opinionated about formatting**, which sounds trivial and is not. There is one correct way to format Go code, a tool called `gofmt` produces it, and every Go project on earth uses it. No Prettier config, no debate about semicolons, no `.editorconfig`. The argument is over because the language ended it.

Go **compiles fast**. Not "fast for a compiled language", fast in absolute terms: a medium project builds in a couple of seconds. This matters more than it sounds, because it means the compile step never becomes a reason to avoid checking your work.

What you give up, stated plainly

A book that only lists advantages is marketing. Here is the other side, and you should weigh it before chapter two.

You will write more lines. Go has no `map`, `filter` or `reduce` built into the language the way JavaScript does. Where you write one chained expression, you will often write a four-line loop. Go's position is that the loop is easier to read six months later. You may disagree, and you will still write the loop.

Error handling is verbose. You are about to meet `if err != nil` and you will type it hundreds of times. There is no `try` / `catch` wrapping half a function. Every single

operation that can fail says so, at the point it can fail. This is the single biggest adjustment coming from JavaScript, and chapter four is entirely about making peace with it.

The ecosystem is smaller. npm has over two million packages. Go has far fewer. For web application work this matters less than you would think, because Go's standard library covers an enormous amount of what you would reach for a package to do in Node. But if you need an obscure SDK, check before you commit.

No null safety. Go has `nil`, and a nil pointer dereference will crash your program at runtime exactly like reading a property of `undefined` crashes yours. Go's type system is simpler than TypeScript's, and in this one specific area TypeScript is genuinely ahead.

> **WATCH OUT**
> If someone tells you Go is simply better than JavaScript, stop listening to them. The two languages were built for different jobs by people with different problems. Go's concurrency is better than the event loop for sustained work, and the event loop is a beautiful fit for a UI. Use both.

The one example that explains everything

Here is the same program in both languages: read three URLs, and print how long each took. It is worth studying before you understand any Go syntax, because the difference in shape is the whole argument.

JAVASCRIPT

```
async function timeAll(urls) {
  const results = await Promise.all(
    urls.map(async (url) => {
      const start = Date.now();
      await fetch(url);
      return { url, ms: Date.now() - start };
    })
  );
  for (const r of results) {
    console.log(r.url, r.ms);
  }
}
```

GO

```
func timeAll(urls []string) {
    var wg sync.WaitGroup
    results := make([]result, len(urls))

    for i, url := range urls {
```

```
        wg.Add(1)
        go func() {
            defer wg.Done()
            start := time.Now()
            http.Get(url)
            results[i] = result{url, time.Since(start)}
        }()
    }

    wg.Wait()
    for _, r := range results {
        fmt.Println(r.url, r.ms)
    }
}
```

The JavaScript is shorter and, if you know JavaScript, clearer. Do not let anybody tell you otherwise.

But look at what the Go version is doing. The word `go` in front of that function call starts a **goroutine**: an independent line of execution that the Go runtime schedules onto a real operating system thread. Three URLs here, but the same code with fifty thousand URLs would start fifty thousand goroutines, and Go would handle it, because a goroutine costs about two kilobytes of memory rather than the megabyte an OS thread costs.

The JavaScript version is also concurrent, and for three HTTP requests it is just as fast, because all three are waiting on the network and the event loop is excellent at waiting. The difference appears when the work is not waiting. If each of those tasks had to parse a large document or resize an image, the JavaScript version would do them one at a time on one core, and the Go version would use every core you have.

> **THE MENTAL MODEL SHIFT**
> In JavaScript, concurrency is something you **await**, and parallelism is something you get by starting more processes. In Go, concurrency is something you **start**, with one keyword, and parallelism comes free because the runtime spreads goroutines across cores. Chapter eleven is where this clicks properly. For now, notice only that `go doSomething()` is the entire syntax.

What this book does

This book has two halves and they have different jobs.

Part one through three teach you Go, assuming you know JavaScript and nothing else. Every concept is introduced by naming the JavaScript thing it resembles, then

showing exactly where the resemblance breaks. You will build small things constantly: a CLI tool, a JSON API with no framework at all, a URL shortener backed by a real database. By the end you can read and write Go without this book open.

Part four and five teach you Grit, which is a framework that generates a Go API, a React admin panel and the typed client between them from one description of your data. The reason it comes second is that a framework you cannot read is a liability. Once you know Go, Grit's generated code is just Go you did not have to type, and you can change any of it.

> **WHAT YOU WILL HAVE BUILT BY THE END**
> A command line tool that formats JSON. An HTTP API written with nothing but the standard library. A URL shortener with a SQLite database. A task tracker, a blog with a working admin panel, and an invoicing application with customers, line items and money handled correctly. All of them small, all of them finished, none of them a toy that stops at the interesting part.

How to read it

Type the code. Do not copy and paste it. This is advice everybody gives and almost nobody follows, so here is the specific reason it matters for Go: the compiler's error messages are a large part of how you learn the language, and you only see them if you make mistakes. Typing produces mistakes. Pasting does not.

Run everything. Every code block that is a complete program is marked, and you should run it, break it deliberately, and run it again. Go builds in seconds, so the cost of an experiment is almost nothing.

Do the "Your turn" boxes. They are short, and they are the difference between recognising Go and writing it.

> **YOUR TURN**
> Before the next chapter, answer this for your own situation in two sentences: what is the specific thing in your current stack that made you pick this book up? Write it down. In chapter twenty three there is a list of twenty six problems that every project runs into, and you will want to know which of them is yours.

0

Installing Go and your first program

This chapter gets Go on your machine, gets a program running, and introduces the toolchain. If you have ever set up a Node project you will find this shorter than you expect, because most of the steps you are used to do not exist.

Installing

Go to go.dev/dl and download the installer for your platform. There is no version manager to install first, no equivalent of `nvm`, and for most people there never needs to be. Go's compatibility promise means a program written for Go 1.0 in 2012 still compiles today, so "which version" is rarely a question that costs you an afternoon.

Check it worked:

```
go version
```

You should see something like `go version go1.25.0 windows/amd64`. If the command is not found, the installer did not put Go on your `PATH`; restart your terminal first, because that fixes it more often than anything else.

> **IF YOU KNOW JAVASCRIPT**
> There is no `nvm`, no `.nvmrc`, and no per-project runtime. One Go installation builds every project on your machine. When a project needs a specific version, it says so in one line of `go.mod` and the toolchain downloads that compiler itself.

Your first program

Make a directory and put one file in it.

```
// main.go
package main

import "fmt"

func main() {
```

```
    fmt.Println("Hello from Go")
}
```

Run it:

```
go run main.go
```

Four things in that tiny file deserve explanation, because each is a rule you will meet constantly.

`package main` declares which package this file belongs to. Every Go file starts with a package declaration. The name `main` is special: it means this package builds into a runnable program rather than a library.

`import "fmt"` pulls in the formatting package from the standard library. Note there is no `npm install`. `fmt` ships with Go, as does an HTTP server, a JSON encoder, a test runner, a crypto suite and a templating engine.

`func main()` is the entry point. A program starts here. There is exactly one `main` function in a program.

`fmt.Println` prints a line. Capital `P` matters enormously, and it is the next section.

Capital letters are the access modifier

This is the first genuinely surprising rule in Go, and it catches everybody.

An identifier starting with a capital letter is exported. One starting with a lowercase letter is private to its package. That is the entire visibility system. There is no `public`, no `private`, no `export` keyword.

```
package money

// Convert is callable from other packages, because it is capitalised.
func Convert(cents int) string { ... }

// round is not. Only code inside package money can call it.
func round(f float64) int { ... }
```

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript you choose what leaves a module with `export`. In Go the name itself decides. `fmt.Println` works because `Println` is capitalised inside package `fmt`; if it were `println` you could not reach it. This applies to functions, types, struct fields, constants and variables alike.

WATCH OUT
This bites hardest with struct fields and JSON. A lowercase field is invisible to the JSON encoder, because the encoder lives in another package and genuinely cannot see it. A struct whose fields are all lowercase encodes to `{}`, with no error and no warning. Chapter sixteen covers this properly, but remember the symptom now: empty JSON usually means lowercase fields.

Modules: go.mod is your package.json

A single file is fine for a demo. A real project is a **module**. Initialise one:

```
mkdir hello && cd hello
go mod init github.com/yourname/hello
```

That writes `go.mod`:

```
module github.com/yourname/hello

go 1.25
```

That is the whole file. Compare it to a `package.json` for an equivalent project and notice what is missing: no `scripts`, no `devDependencies`, no `main`, no `type: module`. The module path doubles as the import path, which is why it looks like a URL: if you push this to GitHub, anybody can import it at exactly that address with no registry publish step.

Add a dependency and `go.mod` grows a line:

```
go get github.com/google/uuid
```

```
module github.com/yourname/hello

go 1.25

require github.com/google/uuid v1.6.0
```

IF YOU KNOW JAVASCRIPT
`go.sum` is your lockfile, equivalent to `package-lock.json`. Commit it. There is no `node_modules`: downloaded packages go to a shared module cache outside your project, and the compiler links what it needs directly into the binary. Deleting a project directory does not take 300MB of dependencies with it, because they were never there.

The commands you will actually use

Six commands cover almost everything.

`go run .`	Compile and run the current package, without keeping the binary
`go build`	Compile to an executable in the current directory
`go test ./...`	Run every test in this module
`go mod tidy`	Add what is imported, remove what is not. Run it often
`gofmt -w .`	Format every file. Your editor probably does this on save
`go vet ./...`	Report suspicious code the compiler allows

The `./...` pattern means "this directory and everything under it", and you will type it constantly.

Compiling a real binary

This is the part that sells Go to people who have deployed Node.

```
go build -o hello .
./hello
```

You now have one executable file. It contains your code, every dependency, and the Go runtime. The target machine needs nothing installed. You can also build for a platform you are not on:

```
GOOS=linux GOARCH=amd64 go build -o hello-linux .
```

That produces a Linux binary from a Mac or Windows laptop, with no container, no cross-compilation toolchain and no configuration. Copy it to a Linux server and it runs.

PROJECT 1: A GREETING WITH ARGUMENTS

Write a program that takes a name as a command line argument and greets it, falling back to "world" when none is given. You need `os.Args`, which is a slice of strings where `os.Args[0]` is the program name itself.

```
package main

import (
    "fmt"
    "os"
)
```

```go
func main() {
    name := "world"
    if len(os.Args) > 1 {
        name = os.Args[1]
    }
    fmt.Printf("Hello, %s\n", name)
}
```

Build it, run `./hello Ada`, then run it with no argument. Then break it deliberately: change `len(os.Args) > 1` to `len(os.Args) > 0` and run it with no argument to see what a real Go panic looks like. Reading panics early is worth more than avoiding them.

YOUR TURN

Extend the greeting to accept several names and greet each on its own line. You will need a `for` loop over a slice, which looks like `for _, n := range os.Args[1:] { ... }`. The next chapter explains every piece of that line, so guessing now is fine and the compiler will tell you when you are wrong.

0

Variables, types, and the compiler that argues with you

This is the chapter where Go feels most foreign, and it is worth knowing why in advance. In JavaScript, a variable is a box that holds anything and can hold something else tomorrow. In Go, a variable has one type forever, decided when it is created, and the compiler refuses to build your program if you break that promise.

That refusal is the feature. Work with it for a week and you will notice a whole category of bug has stopped happening to you.

Declaring things

Go has two ways to make a variable, and you will use one of them ninety percent of the time.

```
var name string = "Ada"   // the long form, type stated
var age = 36              // type inferred, still a var
name := "Ada"             // short form: declare and infer, inside
functions
```

The `:=` operator is the one you want. It declares a new variable and infers its type from the value on the right. It only works inside a function, which is why package level variables use `var`.

> **IF YOU KNOW JAVASCRIPT**
> `:=` is closest to `let` in that it creates a new binding. There is no equivalent of `const` for variables created with `:=`. Go's `const` exists, but it is only for values known at compile time: numbers, strings, booleans. You cannot declare a constant struct or slice. If you want "do not reassign this", Go's answer is to write a small enough function that it is obvious.

An unused variable is a **compile error**, not a warning.

```
func main() {
    count := 10
```

```
    fmt.Println("hello")
}
// ./main.go:4:5: declared and not used: count
```

This feels hostile on day one and reasonable by day ten. Dead variables are usually the residue of a half-finished edit, and Go would rather stop you than let the residue accumulate. The same rule applies to unused imports.

> **WATCH OUT**
> The escape hatch is `_`, the blank identifier, which means "I know there is a value here and I am deliberately ignoring it". You will use it constantly in loops and multiple returns. Do not reach for it to silence a real warning: `_ = count` compiles and tells the next reader nothing.

The types you will actually use

Go has a lot of numeric types. You will use about four of them.

Type	JavaScript	Notes
`string`	`string`	Immutable, UTF-8 bytes. Indexing gives a byte, not a character
`int`	`number`	Whole numbers. 64 bit on any machine you own
`float64`	`number`	Decimals. The default for a fractional literal
`bool`	`boolean`	`true` or `false`. No truthiness, ever
`byte`	-	Alias for `uint8`. A byte of data
`rune`	-	Alias for `int32`. One Unicode code point

The one to notice is that JavaScript's single `number` splits into `int` and `float64`, and **Go will not convert between them for you**.

```
count := 10          // int
price := 2.5         // float64
total := count * price
// invalid operation: count * price (mismatched types int and float64)
```

You must say what you mean:

```
total := float64(count) * price   // 25
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript coerces almost anything: `"3" * 2` is `6`, `[] + {}` is a string, and `0.1 + 0.2` is famously not `0.3`. Go does none of this. There is no implicit conversion between any two types, not even `int` and `int64`. Every conversion is written out, which is more typing and removes an entire family of surprise.

Zero values: there is no undefined

Every type in Go has a **zero value**, and a declared variable always has it. There is no `undefined`, and no uninitialised memory.

`string`	`""`
`int`, `float64`	`0`
`bool`	`false`
pointers, slices, maps, interfaces, functions	`nil`

```
var name string
var count int
var ready bool
fmt.Println(name == "", count == 0, ready == false) // true true true
```

This is genuinely useful. A struct you create is immediately valid, with every field at a sensible default, and you never check whether a field "exists".

> **WATCH OUT**
> It is also the one place zero values hurt: you cannot tell "the caller set this to zero" apart from "the caller did not set it". If that distinction matters, for example a form field where empty and absent mean different things, you need a pointer (`*int`, where `nil` means absent) or a dedicated type. Chapter eight covers pointers properly. This problem is real, and it is the usual reason you see `*string` in a struct.

No truthiness

An `if` in Go takes a `bool`. Not a value that is bool-ish. A `bool`.

JAVASCRIPT

```
if (name) {
  greet(name);
}
if (items.length) {
```

```
  render(items);
}
```

GO

```go
if name != "" {
    greet(name)
}
if len(items) > 0 {
    render(items)
}
```

More characters, and no memorising which values are falsy. There is no `==` and `===` distinction either, because there is only one `==` and it does not coerce.

Strings are bytes, and that matters

A Go string is a read-only slice of bytes holding UTF-8. Indexing it gives you a byte.

```go
s := "héllo"
fmt.Println(len(s))     // 6, not 5: é takes two bytes
fmt.Println(s[1])       // 195, a byte value, not "é"
```

To work with characters, range over the string, which decodes runes:

```go
for i, r := range "héllo" {
    fmt.Printf("%d: %c\n", i, r)
}
// 0: h
// 1: é      <- index jumps to 3 next, because é was two bytes
// 3: l
// 4: l
// 5: o
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript strings are UTF-16 and have the same class of problem: `"👍".length` is `2`. Both languages make you think about encoding the moment you leave ASCII. Go is at least explicit that you are holding bytes.

Strings are immutable, so building one in a loop with `+=` allocates a new string every time. For anything beyond a few concatenations use a builder:

```go
var b strings.Builder
for _, word := range words {
```

```
    b.WriteString(word)
    b.WriteString(" ")
}
result := b.String()
```

Control flow, briefly

`if` and `for` are almost familiar, with two differences worth stating.

There are no parentheses around the condition, and braces are mandatory even for one line.

```
if count > 10 {
    fmt.Println("plenty")
} else if count > 0 {
    fmt.Println("some")
} else {
    fmt.Println("none")
}
```

`for` is the only loop keyword. It covers every case JavaScript spreads across `for`, `while` and `for...of`.

```
for i := 0; i < 10; i++ { }      // classic
for count < 10 { }               // while
for { }                          // infinite
for i, v := range items { }      // for...of, with the index
```

An `if` can carry a statement before the condition, scoped to the `if`. You will see this constantly in real code:

```
if err := doSomething(); err != nil {
    return err
}
// err does not exist here, which is the point
```

IF YOU KNOW JAVASCRIPT

`switch` in Go needs no `break`: cases do not fall through. You can switch on no value at all, which makes it a tidy replacement for a long `else if` chain:

```
switch {
case score > 90:
    grade = "A"
case score > 80:
```

```
    grade = "B"
default:
    grade = "C"
}
```

PROJECT 2: A TEMPERATURE TABLE

Write a program that prints a table converting Celsius to Fahrenheit from minus 40 to 100 in steps of 20. The formula is `f = c*9/5 + 32`.

```
package main

import "fmt"

func main() {
    fmt.Printf("%8s %8s\n", "C", "F")
    for c := -40; c <= 100; c += 20 {
        f := float64(c)*9/5 + 32
        fmt.Printf("%8d %8.1f\n", c, f)
    }
}
```

Two things to notice. `float64(c)` is required: without it you get integer division and wrong answers for anything not divisible by five, which is the bug this chapter exists to prevent. And `%8.1f` means "right align in eight columns, one decimal place", which is how Go does string formatting.

Now break it on purpose: remove the `float64(...)` conversion and read the error. Then change it to `c*9/5` with `c` still an `int` and see that it compiles but produces wrong numbers. The compiler catches the type error. It cannot catch the maths error, and knowing which is which is the skill.

YOUR TURN

Add a third column showing Kelvin (`k = c + 273.15`), and right align the header over it. Then make the step size a variable at the top of `main` and confirm the table still lines up when you change it to 10.

0

Functions, multiple returns, and the end of try/catch

This is the chapter that decides whether you get on with Go. Error handling is where the language is most unlike JavaScript, it is the thing newcomers complain about loudest, and the complaint is half right. Let us do the mechanics first and the argument second.

Functions

```
func add(a int, b int) int {
    return a + b
}

func add(a, b int) int {   // same thing, shared type
    return a + b
}
```

The type comes **after** the name, which reads backwards for about a day and then stops mattering. The reason is that it makes complicated declarations read left to right instead of spiralling outwards, which anyone who has read a C function pointer declaration will appreciate.

Functions are values, so the things you do in JavaScript still work:

```
double := func(n int) int { return n * 2 }

func apply(nums []int, f func(int) int) []int {
    out := make([]int, len(nums))
    for i, n := range nums {
        out[i] = f(n)
    }
    return out
}

apply([]int{1, 2, 3}, double)  // [2 4 6]
```

> **IF YOU KNOW JAVASCRIPT**
> That `apply` is `Array.prototype.map`, written by hand. Go had no generics until 1.18 and still has no `map`, `filter` or `reduce` in the standard library for slices. You write the loop. Go's argument is that a three-line loop needs no explanation and a chain of five callbacks sometimes does. You are allowed to find this annoying.

Two return values, and why

A Go function can return several values, and this is not a curiosity: it is the foundation of the error system.

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("divide by zero")
    }
    return a / b, nil
}
```

The caller gets both:

```
result, err := divide(10, 2)
if err != nil {
    fmt.Println("failed:", err)
    return
}
fmt.Println(result)
```

The convention is absolute, across the standard library and every package you will use: **the error is the last return value, and you check it immediately**.

> **IF YOU KNOW JAVASCRIPT**
> JavaScript has one return value, so a function that can fail either throws or returns something like `{ data, error }`. Go makes the second shape the language default. If you have written Go-style error handling in TypeScript with a `Result` type, or used Rust, you already have the mental model.

`error` is just an interface

There is nothing magic about errors. `error` is an interface with one method:

```
type error interface {
    Error() string
}
```

Anything with an `Error() string` method is an error. That means you can make your own, carrying whatever fields you want:

```
type ValidationError struct {
    Field   string
    Message string
}

func (e ValidationError) Error() string {
    return e.Field + ": " + e.Message
}

func validate(age int) error {
    if age < 0 {
        return ValidationError{Field: "age", Message: "must not be
negative"}
    }
    return nil
}
```

Wrapping: adding context without losing the original

A bare error from deep in a call stack is useless on its own. `fmt.Errorf` with the `%w` verb wraps one error inside another, keeping both.

```
func loadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("loading config from %s: %w", path, err)
    }
    ...
}
```

The message becomes a trail that reads from the outside in:

```
loading config from /etc/app.json: open /etc/app.json: no such file or
directory
```

Use `%w`, not `%v`. `%v` formats the error as text and throws the original away, and then both of the checks below stop working.

Two functions read a wrapped chain. `errors.Is` asks "is this error, anywhere in the chain, that specific one":

```
if errors.Is(err, os.ErrNotExist) {
    // the file was missing, however deeply that was wrapped
}
```

`errors.As` asks "is there an error of this type in the chain", and gives it to you with its fields intact:

```
var ve ValidationError
if errors.As(err, &ve) {
    fmt.Println("bad field:", ve.Field)
}
```

WATCH OUT

`err != nil` is the only check that matters, and comparing an error to `nil` has one notorious trap. A function returning a **concrete** error type rather than the `error` interface can produce an error value that is not `nil` even when it holds nothing:

```
func bad() error {
    var e *ValidationError   // nil pointer
    return e                 // not nil! it is a non-nil interface holding a
nil pointer
}
```

The fix is a rule: **always declare the return type as `error`, and return a literal `nil`, never a typed nil variable.** This catches experienced people and it is worth reading twice.

The argument about verbosity

Here is the complaint, stated fairly. This:

JAVASCRIPT

```
async function setup() {
  try {
    const cfg = await loadConfig();
    const db = await connect(cfg.dsn);
    await migrate(db);
    return db;
  } catch (e) {
    throw new Error(`setup failed: ${e.message}`);
  }
}
```

GO

```
func setup() (*DB, error) {
    cfg, err := loadConfig()
    if err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    db, err := connect(cfg.DSN)
    if err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    if err := migrate(db); err != nil {
        return nil, fmt.Errorf("setup: %w", err)
    }
    return db, nil
}
```

The Go is twice as long. That is not a matter of opinion.

What you get for it is that every failure point is visible at the point it can fail. In the JavaScript, which of the three awaited calls threw? You cannot tell from the code, and the catch block treats all three identically whether or not that is what you wanted. In the Go, each failure is a separate decision, and you could handle the migration failure differently from the connection failure by changing one block.

The honest summary: `try` / `catch` is better when you want to treat a whole region of code as one unit that either works or does not. Explicit returns are better when failures deserve individual handling. Most real backend code is the second kind more often than people expect, and Go bet the language on it.

> **WHAT YOU LOSE, AND SHOULD KNOW ABOUT**
> In JavaScript an uncaught exception unwinds to the nearest handler, so a forgotten `catch` still surfaces somewhere. In Go a forgotten error check silently continues with a zero value. Nothing warns you.
>
> This is a real hazard and the tooling answers it: `errcheck`, bundled into `golangci-lint`, fails the build on an unchecked error. Install it early. It is the one linter that is not optional.

Named returns and `defer`

A function can name its return values, which is occasionally useful and frequently overused:

```
func split(sum int) (x, y int) {
    x = sum * 4 / 9
```

```
    y = sum - x
    return          // a "naked" return, sending back x and y
}
```

Use naked returns only in short functions. In a long one, a bare `return` forces the reader to scroll up to learn what is being returned.

Named returns do earn their place with `defer`, which schedules a call to run when the surrounding function exits, no matter how it exits:

```
func process(path string) (err error) {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()     // runs on every return path below

    // ... many returns, all of which close the file
    return nil
}
```

`defer f.Close()` immediately after a successful open is one of the most common patterns in Go, and it is how the language avoids the cleanup bugs that `try` / `finally` exists to solve.

WATCH OUT

Deferred calls run in last-in, first-out order, and their **arguments** are evaluated immediately, not at exit:

```
for i := 0; i < 3; i++ {
    defer fmt.Println(i)   // prints 2, 1, 0
}
```

And a `defer` inside a loop does not run at the end of each iteration, it runs when the **function** returns. Opening a thousand files in a loop with `defer f.Close()` keeps all thousand open until the function ends. Put the body in its own function if you need per-iteration cleanup.

PROJECT 3: A WORD COUNTER THAT HANDLES FAILURE

Write a program that takes a filename, counts the words in it, and reports errors properly rather than crashing.

```
package main

import (
    "fmt"
    "os"
    "strings"
)

func countWords(path string) (int, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return 0, fmt.Errorf("counting words in %s: %w", path, err)
    }
    return len(strings.Fields(string(data))), nil
}

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: wc <file>")
        os.Exit(1)
    }
    n, err := countWords(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
    fmt.Printf("%d words\n", n)
}
```

Run it on a real file, then on one that does not exist, and read the wrapped message. Notice `os.Exit(1)` and writing to `os.Stderr`: a command line tool that reports failure on stdout with exit code zero is a tool that breaks every script that uses it.

`strings.Fields` splits on any run of whitespace, which is the right behaviour here and is worth remembering: it is not the same as `strings.Split(s, " ")`, which gives empty strings between consecutive spaces.

YOUR TURN

Add a check that reports a clearer message when the path is a directory rather than a file. `os.ReadFile` returns an error you can inspect; try `errors.Is(err, syscall.EISDIR)` on Unix, or simply call `os.Stat` first and check `info.IsDir()`, which is portable and is the version you should probably ship.

0

Structs and methods, against objects and classes

A JavaScript object is a bag of keys you can add to at any time. A Go struct is a fixed set of named fields with fixed types, decided when you declare the type and never changing. Everything else in this chapter follows from that one difference.

Declaring and creating

```
type User struct {
    ID    int
    Name  string
    Email string
    Admin bool
}
```

There are four ways to make one, and you will see all four in real code.

```
// 1. Field names. Use this one. It survives a field being added.
u := User{ID: 1, Name: "Ada", Email: "ada@example.com"}

// 2. Positional. Fragile: adding a field breaks every call site.
u := User{1, "Ada", "ada@example.com", false}

// 3. Zero value, then assign.
var u User
u.Name = "Ada"

// 4. new(), giving a pointer to a zero value.
u := new(User)   // *User
```

Notice what is missing from the first example: `Admin` was not set, so it is `false`. There is no `undefined`. Every field always has a value, and a struct you create is immediately complete.

> **IF YOU KNOW JAVASCRIPT**

> The closest JavaScript equivalent is a class with a constructor that sets every property. The difference is that you cannot add a field later. `u.Age = 36` on a `User` with no `Age` field is a compile error, not a new property. This is the whole point: the shape of your data is checked, everywhere, for free.

Methods: functions with a receiver

Go has no classes. You attach behaviour to a type by declaring a function with a **receiver**, the parameter in parentheses before the name.

```
func (u User) DisplayName() string {
    if u.Name == "" {
        return u.Email
    }
    return u.Name
}

fmt.Println(u.DisplayName())
```

`u User` is the receiver. It behaves like a parameter, because that is exactly what it is: `u.DisplayName()` is sugar for passing `u` as the first argument.

> **IF YOU KNOW JAVASCRIPT**
> There is no `this`, and that removes an entire class of JavaScript bug. The receiver is a normal named parameter, bound at the call site, and it cannot be rebound by how the function was called. No `.bind(this)`, no arrow function workaround, no `const self = this`.

Value receivers and pointer receivers

This is the first thing in Go that genuinely trips people up, and it is worth getting right now rather than debugging later.

A **value receiver** gets a copy. Changes to it do not affect the original.

```
func (u User) Rename(name string) {
    u.Name = name       // modifies the copy, then throws it away
}

u := User{Name: "Ada"}
u.Rename("Grace")
fmt.Println(u.Name)     // still "Ada"
```

A **pointer receiver** gets the address, so changes stick.

```
func (u *User) Rename(name string) {
    u.Name = name       // modifies the original
}

u := User{Name: "Ada"}
u.Rename("Grace")
fmt.Println(u.Name)     // "Grace"
```

You do not write `(&u).Rename(...)`. Go inserts the address-of automatically when the method needs a pointer and you have an addressable value.

> **WATCH OUT**
> The rule in practice: **if a method modifies the receiver, it must be a pointer receiver.** A value receiver that assigns to a field compiles happily and silently does nothing, which is one of the few truly quiet bugs in Go.
>
> The second rule: **be consistent within a type.** If any method on `User` takes a pointer, give them all pointers. Mixing the two is legal and makes the rules about which calls are addressable surface at the worst moment.

A practical reason to prefer pointer receivers even when not mutating: a value receiver copies the whole struct on every call. For a struct with twenty fields called in a loop, that is real work for no benefit.

Composition instead of inheritance

Go has no `extends`. It has **embedding**, which looks similar and behaves differently in one important way.

```
type Timestamps struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

type Post struct {
    Timestamps          // embedded: no field name
    Title string
    Body  string
}

p := Post{Title: "Hello"}
p.CreatedAt = time.Now()        // promoted, as if declared on Post
fmt.Println(p.Timestamps.CreatedAt)  // the long way, same field
```

The embedded type's fields and methods are **promoted** to the outer type. If `Timestamps` had a method `Age()`, then `p.Age()` would work.

> **IF YOU KNOW JAVASCRIPT**
> This resembles `class Post extends Timestamps`, and the difference matters: there is no polymorphism here. A `Post` is **not** a `Timestamps` and cannot be passed where one is expected. Embedding is a convenience for forwarding fields and methods, not a type relationship. In Go, the thing that lets one type stand in for another is an interface, which is chapter ten.

Struct tags: metadata for encoders

A field can carry a string of metadata, used by libraries through reflection. You will meet this constantly, because it is how Go does JSON.

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
    Admin bool   `json:"-"`                  // never encoded
    Bio   string `json:"bio,omitempty"`      // dropped when empty
}
```

Without tags, `json.Marshal` uses the Go field names, so you get `{"ID":1, "Name":"Ada"}`. With them you get the lowercase JSON your frontend expects.

> **WATCH OUT**
> Those backticks are a raw string literal, and the format inside is strict: `json:"name"` with no space after the colon. `json: "name"` silently does nothing, the tag is ignored, and the field encodes as `Name`. There is no error. If a field is coming out with the wrong key, check for that space first.

Constructors are just functions

Go has no `constructor` keyword. The convention is a package-level function named `New` or `NewThing` that returns the type.

```
func NewUser(name, email string) (*User, error) {
    if email == "" {
        return nil, errors.New("email is required")
    }
    return &User{
```

```
        Name:  name,
        Email: email,
    }, nil
}
```

Returning `*User` rather than `User` is the usual choice, because the caller almost always wants to pass it around without copying, and because it lets you return `nil` on failure.

> **IF YOU KNOW JAVASCRIPT**
> A JavaScript constructor cannot fail cleanly: it either throws or returns a half-built object. A Go constructor is an ordinary function, so it returns an error like everything else, and you cannot get a `*User` back unless it succeeded.

Comparing structs

Structs are comparable with `==` when all their fields are, and the comparison is by value, field by field.

```
a := User{ID: 1, Name: "Ada"}
b := User{ID: 1, Name: "Ada"}
fmt.Println(a == b)     // true
```

That is deep equality for free, and it is a genuine surprise if you are used to `a === b` on objects being an identity check.

> **WATCH OUT**
> It stops working the moment a struct contains a slice, a map or a function, because those are not comparable. The compiler tells you. For those, use `reflect.DeepEqual`, or in tests `google/go-cmp`, which prints a readable diff instead of just `false`.

> **PROJECT 4: A CONTACT BOOK IN MEMORY**
> Build a small type with behaviour, and exercise value and pointer receivers.
>
> ```
> package main
>
> import (
>     "errors"
>     "fmt"
>     "strings"
> )
>
> type Contact struct {
> ```

```
    Name  string
    Email string
    Tags  []string
}

// Value receiver: reads only, so a copy is fine.
func (c Contact) Display() string {
    if len(c.Tags) == 0 {
        return c.Name
    }
    return fmt.Sprintf("%s [%s]", c.Name, strings.Join(c.Tags, ", "))
}

// Pointer receiver: it modifies, so it must be a pointer.
func (c *Contact) Tag(tag string) {
    for _, t := range c.Tags {
        if t == tag {
            return // already there, do nothing
        }
    }
    c.Tags = append(c.Tags, tag)
}

type Book struct {
    contacts []Contact
}

func (b *Book) Add(c Contact) error {
    if c.Email == "" {
        return errors.New("contact needs an email")
    }
    b.contacts = append(b.contacts, c)
    return nil
}

func (b *Book) Find(email string) (*Contact, bool) {
    for i := range b.contacts {
        if b.contacts[i].Email == email {
            return &b.contacts[i], true   // address of the real one
        }
    }
    return nil, false
}

func main() {
    var book Book
    if err := book.Add(Contact{Name: "Ada", Email: "ada@example.com"});
```

```
    err != nil {
            fmt.Println(err)
            return
        }

        if c, ok := book.Find("ada@example.com"); ok {
            c.Tag("maths")
            c.Tag("maths") // ignored, already present
        }

        for _, c := range book.contacts {
            fmt.Println(c.Display())  // Ada [maths]
        }
    }
```

The subtle line is `return &b.contacts[i], true`. Ranging with `for _, c := range b.contacts` would give you a **copy** of each contact, and tagging that copy would change nothing. Indexing with `i` and taking the address reaches the real element. Change it to the copying version and watch the tag vanish: that is the lesson of this chapter in one experiment.

YOUR TURN

Add a `Remove(email string) bool` method to `Book`. You will need to build a new slice or use the `slices.Delete` helper from the standard library. Then add a `Count()` method and decide, with a reason you can say out loud, whether it takes a value or a pointer receiver.

0

Slices: the array that is not an array

Go has arrays and it has slices. You will use slices for essentially everything, and the distinction matters exactly twice: when you declare one, and when a surprising bug appears. This chapter covers both.

Arrays have a fixed length, in the type

```
var a [3]int        // an array of exactly three ints
a[0] = 1
fmt.Println(len(a)) // 3
```

The length is **part of the type**. `[3]int` and `[4]int` are different types, and a function taking `[3]int` will not accept `[4]int`. Arrays are also copied whole on assignment. This is almost never what you want, which is why arrays are rare in Go code.

Slices are a view onto an array

```
nums := []int{1, 2, 3}      // no length in the brackets: a slice
nums = append(nums, 4)
fmt.Println(len(nums))      // 4
```

A slice is a small three-field value: a pointer to an underlying array, a length, and a capacity. That is the whole model, and understanding it explains every slice surprise you will ever hit.

`len(s)`	How many elements you can index right now
`cap(s)`	How many fit before the underlying array must be reallocated

```
s := make([]int, 2, 5)   // len 2, cap 5
fmt.Println(len(s), cap(s))  // 2 5
s = append(s, 10)            // fits in the existing array
fmt.Println(len(s), cap(s))  // 3 5
```

`make([]T, len, cap)` creates one with room reserved. When you know roughly how many elements are coming, say so: it avoids repeated reallocation.

> **IF YOU KNOW JAVASCRIPT**
> A JavaScript array handles all of this invisibly. `push` grows it and you never think about capacity. Go exposes the machinery, which costs you some attention and buys you control over allocation, which matters a great deal in a hot loop and not at all in a handler that runs once per request.

append, and the thing it does not promise

`append` returns a slice. **You must use the return value.**

```
nums := []int{1, 2, 3}
append(nums, 4)          // compile error: result not used
nums = append(nums, 4)   // correct
```

The reason is that `append` may or may not allocate a new array. If there is spare capacity it writes in place and returns a slice with a longer length. If there is not, it allocates a bigger array, copies everything over, and returns a slice pointing at the **new** array. The caller's original slice still points at the old one.

This is where the famous slice bug lives.

```
a := []int{1, 2, 3, 4, 5}
b := a[1:3]              // [2 3], sharing a's array
b[0] = 99
fmt.Println(a)           // [1 99 3 4 5]  <- a changed
```

Slicing does not copy. `b` is a window onto the same memory. Writing through one is visible through the other.

> **WATCH OUT**
> It gets worse with `append`, because whether you corrupt the original depends on capacity, which depends on history:
>
> ```
> a := []int{1, 2, 3, 4, 5}
> b := a[1:3]              // len 2, cap 4 (to the end of a)
> b = append(b, 99)        // fits in cap, writes into a's array
> fmt.Println(a)           // [1 2 3 99 5]  <- clobbered a[3]
> ```
>
> The fix, when you want a genuinely independent slice, is a three-index slice that caps capacity, or an explicit copy:

```
b := a[1:3:3]           // cap is now 2, so append must allocate
// or
b := slices.Clone(a[1:3])
```

This bug is rare in everyday handler code and vicious when it appears. The tell is "a value changed and nothing in my code assigned to it".

Iterating

```
for i, v := range nums {
    fmt.Println(i, v)
}

for _, v := range nums {    // index discarded
    fmt.Println(v)
}

for i := range nums {       // value discarded
    nums[i] *= 2
}
```

`range` gives you a **copy** of each element. For ints that is irrelevant. For structs it is the bug from the previous chapter: modifying `v` changes nothing. Use the index when you need to write.

IF YOU KNOW JAVASCRIPT

Go has no `map`, `filter`, `reduce`, `find` or `some` for slices in the language. Since Go 1.21 the `slices` package covers some of it:

```
slices.Contains(nums, 3)        // bool
slices.Index(nums, 3)           // position or -1
slices.Sort(nums)               // in place
slices.Reverse(nums)            // in place
slices.Max(nums)                // and Min
```

For anything else, write the loop. A filter is four lines:

```
var active []User
for _, u := range users {
    if u.Active {
        active = append(active, u)
    }
}
```

Passing slices to functions

A slice is cheap to pass: you are copying three words, not the data. That means a function **can** modify the caller's elements.

```
func zero(nums []int) {
    for i := range nums {
        nums[i] = 0
    }
}

nums := []int{1, 2, 3}
zero(nums)
fmt.Println(nums)   // [0 0 0]
```

But it cannot change the caller's **length**, because the slice header itself was copied:

```
func addOne(nums []int) {
    nums = append(nums, 1)   // changes the local copy's header only
}

nums := []int{1, 2, 3}
addOne(nums)
fmt.Println(len(nums))   // still 3
```

The convention, and the reason `append` works the way it does: a function that grows a slice returns it.

```
func addOne(nums []int) []int {
    return append(nums, 1)
}
nums = addOne(nums)
```

Sorting

```
people := []User{{Name: "Zoe"}, {Name: "Ada"}}

slices.SortFunc(people, func(a, b User) int {
    return strings.Compare(a.Name, b.Name)
})
```

The comparison function returns a negative number, zero, or a positive number, the same contract as JavaScript's `Array.prototype.sort` comparator. For stable sorting use `slices.SortStableFunc`.

PROJECT 5: WORD FREQUENCY, TOP TEN

Read a file, count how often each word appears, and print the ten most common. This uses slices, sorting and a map together, and it is a real tool you can point at a book.

```
package main

import (
    "fmt"
    "os"
    "slices"
    "strings"
)

type pair struct {
    word  string
    count int
}

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: freq <file>")
        os.Exit(1)
    }

    data, err := os.ReadFile(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }

    counts := map[string]int{}
    for _, w := range strings.Fields(string(data)) {
        w = strings.ToLower(strings.Trim(w, ".,!?;:\"'()"))
        if w != "" {
            counts[w]++
        }
    }

    pairs := make([]pair, 0, len(counts))
    for w, c := range counts {
        pairs = append(pairs, pair{w, c})
    }

    slices.SortFunc(pairs, func(a, b pair) int {
        if b.count != a.count {
            return b.count - a.count       // count descending
        }
        return strings.Compare(a.word, b.word) // then alphabetical
```

```
    })

    for _, p := range pairs[:min(10, len(pairs))] {
        fmt.Printf("%6d  %s\n", p.count, p.word)
    }
}
```

Three things worth noticing. `make([]pair, 0, len(counts))` reserves capacity up front because we know exactly how many there will be. The tie-break on word makes the output deterministic, which matters because map iteration order is random and without it two runs would disagree. And `min` is a builtin since Go 1.21, so `pairs[:min(10, len(pairs))]` does not panic on a file with fewer than ten distinct words.

YOUR TURN

Add a flag so the user can ask for a different number of results, for example `freq book.txt 25`. Then add a stop-word list so "the", "a" and "and" are excluded, and decide whether a slice or a map is the better structure for holding those stop words. The next chapter will tell you whether you chose right.

0

Maps, and why iteration order is random on purpose

A Go map is a hash table with typed keys and typed values. It is the closest thing Go has to a JavaScript object used as a dictionary, and it differs in four ways that each cause a specific bug the first time you meet them.

Declaring and using

```
ages := map[string]int{
    "ada":   36,
    "grace": 45,
}

ages["alan"] = 41
fmt.Println(ages["ada"])    // 36
fmt.Println(len(ages))      // 3
delete(ages, "alan")
```

The type is `map[KeyType]ValueType`. Keys must be comparable, so strings, ints, booleans and structs of those are fine; slices, maps and functions are not.

WATCH OUT

A `nil` map is readable but **not writable**, and this is the first map bug everyone hits:

```
var m map[string]int      // nil
fmt.Println(m["a"])       // 0, fine
fmt.Println(len(m))       // 0, fine
m["a"] = 1                // panic: assignment to entry in nil map
```

Always create one before writing: `m := map[string]int{}` or `m := make(map[string]int)`. The same trap hides inside structs, where a map field is nil until you assign one, so a constructor that forgets it produces a struct that panics on first write.

Missing keys return the zero value

Reading a key that is not there is not an error. You get the zero value of the value type.

```
counts := map[string]int{}
fmt.Println(counts["nothing"])   // 0, not undefined
counts["word"]++                 // works: reads 0, writes 1
```

That last line is why counting in Go is so tidy: no "if the key exists, add one, otherwise set it to one".

To tell "absent" from "present but zero", use the two-value form:

```
v, ok := ages["nobody"]
if !ok {
    fmt.Println("not in the map")
}
```

`ok` is a bool. This is the **comma ok** idiom, and you will see the same shape for type assertions and channel receives.

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript, `obj.missing` gives `undefined`, and you distinguish absent from set-to-undefined with `in` or `hasOwnProperty`. Go's `v, ok :=` is the same idea with better ergonomics, because the check and the read are one operation and you cannot forget which you did.

Iteration order is deliberately random

```
for k, v := range ages {
    fmt.Println(k, v)
}
```

Run that twice and the order differs. This is not an implementation accident that might settle down: the runtime randomises the starting point on purpose, so that code which accidentally depends on order fails immediately and loudly rather than in production two years later when the table grows.

To iterate in a stable order, sort the keys:

```
keys := make([]string, 0, len(ages))
for k := range ages {
    keys = append(keys, k)
}
slices.Sort(keys)
```

```
for _, k := range keys {
    fmt.Println(k, ages[k])
}
```

> **IF YOU KNOW JAVASCRIPT**
> JavaScript objects do have an order: integer-like keys ascending, then string keys in insertion order. Plenty of code quietly relies on it. Go removes the temptation. If your output needs an order, you must say what that order is, which is usually an improvement to the code anyway.

Maps are reference types

Passing a map to a function does not copy it. The function can modify the caller's map.

```
func add(m map[string]int) {
    m["new"] = 1        // visible to the caller
}
```

This is unlike a struct and unlike an array, and like a slice's elements. There is no `const` map and no way to pass one read-only. If a function must not modify a map, the only guards are a copy (`maps.Clone`) or a comment.

Structs as values, and the thing you cannot do

A map of structs has one sharp edge:

```
type Stat struct{ Count int }
m := map[string]Stat{"a": {Count: 1}}

m["a"].Count++      // compile error: cannot assign to struct field
```

The map returns a **copy** of the struct, so incrementing it would modify something about to be discarded, and Go refuses rather than letting you write code that does nothing. Two fixes:

```
// 1. Read, modify, write back.
s := m["a"]
s.Count++
m["a"] = s

// 2. Store pointers instead.
```

```
m := map[string]*Stat{"a": {Count: 1}}
m["a"].Count++      // fine
```

The second is usually what you want when the values are large or frequently updated. The first is fine for small structs and avoids a layer of indirection.

Sets

Go has no set type. The idiom is a map with an empty struct value:

```
seen := map[string]struct{}{}
seen["ada"] = struct{}{}

if _, ok := seen["ada"]; ok {
    fmt.Println("already seen")
}
```

`struct{}` is a struct with no fields and it occupies zero bytes, so the map stores only keys. Using `map[string]bool` is also common and slightly more readable at the cost of one byte per entry; both are accepted Go.

> **IF YOU KNOW JAVASCRIPT**
> JavaScript has a real `Set` with `.has()` and `.add()`. Go's version is wordier, and that `struct{}{}` reads as line noise until you have seen it a dozen times. Read it as: the type `struct{}`, then an empty literal of it.

Maps are not safe for concurrent use

Two goroutines writing to the same map at the same time will crash the program with a runtime error, deliberately. Go detects it and stops rather than corrupting the table silently.

The answer is usually a mutex, which is chapter fifteen:

```
type Counter struct {
    mu     sync.Mutex
    counts map[string]int
}

func (c *Counter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.counts[key]++
}
```

IF YOU KNOW JAVASCRIPT
This has no JavaScript equivalent, because the event loop means two pieces of your code never run at literally the same instant. In Go they do. Any value shared between goroutines needs protection, and a map is the one where forgetting is immediately fatal rather than subtly wrong, which is mercifully the better failure.

PROJECT 6: A WORD INDEX WITH LINE NUMBERS
Extend the frequency counter into something genuinely useful: for each word, record every line it appears on. This is a `map[string][]int`, a map whose values are slices, and it is a shape you will use often.

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "slices"
    "strings"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: index <file>")
        os.Exit(1)
    }

    f, err := os.Open(os.Args[1])
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
    defer f.Close()

    index := map[string][]int{}

    scanner := bufio.NewScanner(f)
    line := 0
    for scanner.Scan() {
        line++
        for _, w := range strings.Fields(scanner.Text()) {
            w = strings.ToLower(strings.Trim(w, ".,!?;:\"'()"))
            if len(w) < 4 {
                continue    // skip short words
            }
```

```
            // Appending to a nil slice works, so no "if absent" branch.
            if !slices.Contains(index[w], line) {
                index[w] = append(index[w], line)
            }
        }
    }
    if err := scanner.Err(); err != nil {
        fmt.Fprintln(os.Stderr, "reading:", err)
        os.Exit(1)
    }

    words := make([]string, 0, len(index))
    for w := range index {
        words = append(words, w)
    }
    slices.Sort(words)

    for _, w := range words {
        if len(index[w]) > 2 {   // only words on three or more lines
            fmt.Printf("%-16s %v\n", w, index[w])
        }
    }
}
```

The line that rewards attention is `index[w] = append(index[w], line)`. If `w` is not in the map, `index[w]` is a nil slice, and appending to a nil slice is legal and returns a one-element slice. So the "create the entry if missing" branch that you would write in most languages simply does not exist here. Two zero-value rules, slices and maps, combining to delete a line of code.

Also note `bufio.Scanner`: it reads line by line without loading the file into memory, so this works on a file larger than your RAM, unlike the `os.ReadFile` in the last project. And `scanner.Err()` must be checked, because the loop ends identically on end-of-file and on a read error.

YOUR TURN

Change the value type to a small struct holding both the line numbers and a total count, and watch the compiler stop you from writing `index[w].Count++`. Fix it both ways described above, then decide which you prefer and why. That decision, made consciously once, is the difference between copying Go idioms and understanding them.

0

Pointers, without the fear

Pointers have a reputation they do not deserve, earned in languages where you can do arithmetic on them and walk off the end of memory. Go's pointers cannot do that. There is no pointer arithmetic, and the garbage collector means you never free anything. What is left is a small, useful idea: **a pointer is the address of a value rather than a copy of it**.

You have already been using them. This chapter names what was happening.

Everything in Go is passed by value

This is the single sentence that explains pointers, and it is worth reading twice because it is the opposite of your JavaScript instinct.

When you pass anything to a function, Go copies it. An int, a struct, a slice header, all copied. So a function cannot change what you passed:

```
func rename(u User) {
    u.Name = "changed"      // changes the copy
}

u := User{Name: "Ada"}
rename(u)
fmt.Println(u.Name)         // "Ada"
```

> **IF YOU KNOW JAVASCRIPT**
> In JavaScript, objects are passed by **reference to the same object**, so a function that sets `user.name` changes the caller's object. Primitives are copied. Go removes the distinction: everything is copied, every time, and when you want the other behaviour you ask for it explicitly with a pointer. Less to remember, more to type.

Two operators

```
u := User{Name: "Ada"}

p := &u        // & takes the address: p is a *User
```

```
fmt.Println(*p)     // * dereferences: the User itself
fmt.Println(p.Name) // but you rarely need *, see below
```

`&x` means “the address of x”. `*T` in a type means “pointer to T”. `*p` as an expression means “the value at p”.

Now the function that works:

```
func rename(u *User, name string) {
    u.Name = name
}

rename(&u, "Grace")
fmt.Println(u.Name)     // "Grace"
```

WATCH OUT

You almost never write `(*u).Name`. Go dereferences automatically for field access and method calls, so `u.Name` works whether `u` is a `User` or a `*User`. This is a deliberate convenience and it means pointer-ness mostly disappears from the code that uses the value. The place it reappears is `nil`.

nil, and the panic you will meet

A pointer’s zero value is `nil`. Using a field on a nil pointer crashes:

```
var u *User          // nil
fmt.Println(u.Name)  // panic: runtime error: invalid memory address
                     // or nil pointer dereference
```

This is Go’s `Cannot read properties of undefined`, with the same cause and a louder failure: the program stops rather than continuing with a bad value.

The guard is an ordinary `if`:

```
func display(u *User) string {
    if u == nil {
        return "(none)"
    }
    return u.Name
}
```

IF YOU KNOW JAVASCRIPT

> TypeScript's `strictNullChecks` would make this a compile error. Go has no equivalent: a `*User` might be nil and the compiler will not stop you. This is the one area where TypeScript's type system is genuinely ahead, and it is worth saying rather than glossing over. Your defences in Go are: return values rather than pointers where you can, check at the boundary, and let the panic happen in development rather than coding defensively everywhere.

When to use a pointer

Three reasons, and nothing else.

One: you need to modify the caller's value. This is the common case, and it is why methods that change state take pointer receivers.

Two: the struct is large and copied often. A struct with twenty fields copied in a tight loop is real work. For a three-field struct it does not matter, and chasing this before measuring is how code gets uglier for no gain.

Three: nil is a meaningful value. A `*string` field distinguishes "the user did not supply this" from "the user supplied an empty string". You met this in chapter three as the limit of zero values. This is the fix.

```
type UpdateUser struct {
    Name  *string `json:"name"`   // nil means "leave it alone"
    Email *string `json:"email"`
}

func apply(u *User, patch UpdateUser) {
    if patch.Name != nil {
        u.Name = *patch.Name       // explicitly set, even to ""
    }
    if patch.Email != nil {
        u.Email = *patch.Email
    }
}
```

That is how a PATCH endpoint tells "clear this field" apart from "do not touch this field", and you will write it for real in part four.

When not to use a pointer

Slices, maps and channels already hold an internal pointer. Passing `*[]int` or `*map[string]int` is almost always wrong and marks code as written by someone still finding their feet.

```go
func addItem(items []string) []string {   // correct
    return append(items, "new")
}

func addItem(items *[]string) {           // almost always wrong
    *items = append(*items, "new")
}
```

The slice version returns the new slice, which is the convention `append` itself follows. The pointer version works and reads badly.

Small structs you only read are also better by value. A `Point{X, Y int}` is sixteen bytes; copying it is cheaper than following a pointer.

The loop variable, and a trap that Go fixed

For years this was the most common Go bug in existence:

```go
// Go 1.21 and earlier
for _, u := range users {
    go func() {
        fmt.Println(u.Name)   // every goroutine saw the SAME u
    }()
}
```

The loop variable was reused across iterations, so all the goroutines shared one variable and usually printed the last element several times.

Go 1.22 changed this. Loop variables are now per-iteration, and the code above does what it looks like it does.

> **WATCH OUT**
> You will still find the old workaround everywhere in blog posts and older codebases:
>
> ```go
> for _, u := range users {
>     u := u                // "shadow" it, making a per-iteration copy
>     go func() { ... }()
> }
> ```
>
> It is harmless on modern Go and no longer needed. But check your `go.mod`: the new behaviour only applies if the `go` directive says 1.22 or higher. A module that still says `go 1.21` gets the old semantics **even on a new toolchain**, which is a subtle and genuinely nasty way to inherit this bug.

Pointers to struct fields

You can take the address of a field, and of a slice element, which is how the contact book in chapter five worked:

```
for i := range book.contacts {
    if book.contacts[i].Email == email {
        return &book.contacts[i]    // address of the real element
    }
}
```

What you cannot take the address of is a map entry, because the map may move its contents when it grows:

```
m := map[string]User{"a": {Name: "Ada"}}
p := &m["a"]        // compile error: cannot take address of m["a"]
```

This is the same restriction that stopped `m["a"].Name++` in chapter seven, and it has the same two fixes: read-modify-write, or store `*User` in the map.

PROJECT 7: AN INVENTORY WITH ADJUSTABLE STOCK

Build something where the pointer decisions actually matter, and where getting one wrong is visible.

```
package main

import (
    "errors"
    "fmt"
)

type Item struct {
    SKU   string
    Name  string
    Stock int
}

type Inventory struct {
    items map[string]*Item    // pointers: we mutate these in place
}

func NewInventory() *Inventory {
    return &Inventory{items: map[string]*Item{}}
}

func (inv *Inventory) Add(sku, name string, stock int) {
```

```
    inv.items[sku] = &Item{SKU: sku, Name: name, Stock: stock}
}

// Get returns nil when absent, so callers must check. The bool makes that
// impossible to forget by accident.
func (inv *Inventory) Get(sku string) (*Item, bool) {
    it, ok := inv.items[sku]
    return it, ok
}

var ErrInsufficientStock = errors.New("insufficient stock")

func (inv *Inventory) Take(sku string, n int) error {
    it, ok := inv.items[sku]
    if !ok {
        return fmt.Errorf("taking %s: not found", sku)
    }
    if it.Stock < n {
        return fmt.Errorf("taking %d of %s: %w", n, sku,
ErrInsufficientStock)
    }
    it.Stock -= n       // through the pointer, so the map sees it
    return nil
}

func main() {
    inv := NewInventory()
    inv.Add("A1", "Blue widget", 10)

    if err := inv.Take("A1", 3); err != nil {
        fmt.Println(err)
    }
    if err := inv.Take("A1", 99); err != nil {
        fmt.Println("expected:", err)
        fmt.Println("is stock error:", errors.Is(err, ErrInsufficientStock))
    }

    if it, ok := inv.Get("A1"); ok {
        fmt.Printf("%s: %d left\n", it.Name, it.Stock)   // 7 left
    }
}
```

Now do the experiment that teaches the chapter. Change the map to `map[string]Item`, holding values rather than pointers, and try to compile. The line `it.Stock -= n` will still compile, because `it` is now a copy, and the stock will never change. Run it and watch `10 left` print after you took three. Then make it work by reading, modifying and writing back.

That bug, with values where you meant pointers, is the single most common mistake newcomers make with Go collections, and having caused it once on purpose you will recognise it instantly.

YOUR TURN

Add a `Restock(sku string, n int) error` method, then add a `Low(threshold int) []*Item` that returns every item below a stock level. Decide whether `Low` should return `[]*Item` or `[]Item` and be able to say why. One of them lets a caller change your inventory by accident, which might be exactly what you want or exactly what you do not.

0

Packages, modules and imports, against npm and ESM

You have been writing single-file programs. Real projects are many files, and Go's rules for organising them are stricter and simpler than what you are used to. This chapter is the one that makes a Go repository stop looking strange.

A package is a directory

Every `.go` file belongs to a package, declared on its first line. **Every file in a directory must declare the same package name**, and a directory is exactly one package. That is the whole rule.

```
myapp/
├── go.mod              module github.com/you/myapp
├── main.go             package main
├── store/
│   ├── store.go        package store
│   └── user.go         package store
└── money/
    └── money.go        package money
```

Files in `store/` can see each other's identifiers with no import at all, even the lowercase ones. There is no equivalent of importing between files in the same folder, because they are already the same package.

> **IF YOU KNOW JAVASCRIPT**
> This is the biggest structural difference from JavaScript. In a Next.js project every file is its own module, and you import between them constantly. In Go, `store/store.go` and `store/user.go` share a namespace: a function declared in one is callable from the other with no ceremony. You import across **directories**, never within one.

Importing

```
package main

import (
    "fmt"                // standard library
    "net/http"

    "github.com/google/uuid"          // third party

    "github.com/you/myapp/store"      // your own
)

func main() {
    s := store.New()
    fmt.Println(uuid.NewString())
}
```

You refer to things by the **package name**, which is the last element of the path by convention but is really whatever the files declare. `store.New`, not `New`.

The grouping above is conventional: standard library first, then third party, then your own module, separated by blank lines. `gofmt` sorts within groups and `goimports` adds and removes imports as you type.

> **WATCH OUT**
> An unused import is a compile error, exactly like an unused variable. This is why `goimports` in your editor is not optional: without it you will spend real time adding and removing import lines by hand.
>
> The escape hatch, for a package imported only for its side effects, is the blank identifier:
>
> ```
> import _ "github.com/lib/pq"   // registers a database driver
> ```

The import path is the module path plus the directory

There is no `baseUrl`, no `paths` mapping, no `@/components` alias, and no relative imports. `import "./store"` is not valid Go.

If your module is `github.com/you/myapp`, then the directory `store/` is imported as `github.com/you/myapp/store`. Always the full path.

> **IF YOU KNOW JAVASCRIPT**
> Verbose compared to `@/store`, and it buys one thing: an import path is globally unambiguous. You can paste it into a browser and often land on the source. There is no

build-time configuration that has to agree with your editor, your bundler and your test runner, because there is no mapping at all.

internal/ is a real access rule

A directory named `internal` is enforced by the compiler: only code rooted at its parent may import it.

```
myapp/
├── internal/
│   └── db/            importable by myapp and anything under it
└── pkg/
    └── client/        importable by anybody
```

If someone imports `github.com/you/myapp/internal/db` from a different module, the build fails. This is how a library publishes a small surface while keeping the rest private, and it is why nearly every serious Go project puts most of its code under `internal/`.

IF YOU KNOW JAVASCRIPT

The nearest npm equivalent is the `exports` field in `package.json`, which controls what consumers can reach. Go's version is a directory name, needs no configuration and cannot be worked around.

Naming, and the stutter rule

Package names are short, lowercase, single words. No underscores, no camelCase, no plurals.

The identifier is read **with** the package name, which leads to the most quoted style rule in Go:

```
// Bad: the package name is already there.
store.NewStore()
http.HTTPServer
money.MoneyAmount

// Good:
store.New()
http.Server
money.Amount
```

WATCH OUT

Avoid `util`, `helpers`, `common` and `shared`. They are not packages, they are places things go to hide, and they grow into a dependency that everything imports and nobody understands. If a function does not belong anywhere, that usually means the thing it operates on wants a package of its own.

Modules, versions and go.mod

A module is a tree of packages with one `go.mod` at the root. You saw it in chapter two; here is what the rest of it means.

```
module github.com/you/myapp

go 1.25

require (
    github.com/gin-gonic/gin v1.10.0
    gorm.io/gorm v1.25.12
)

require (
    github.com/bytedance/sonic v1.12.0 // indirect
)
```

The second `require` block is indirect dependencies: things your dependencies need. `go mod tidy` manages both blocks, adding what is imported and removing what is not. Run it before every commit.

IF YOU KNOW JAVASCRIPT

`go.sum` is the lockfile and must be committed. Unlike `package-lock.json` it is not a full dependency tree: it is a list of cryptographic hashes. Go uses **minimal version selection**, meaning it picks the lowest version that satisfies everyone, so builds are reproducible from `go.mod` alone and the sum file exists to verify integrity rather than to pin.

There is also no `devDependencies`. A tool you use at development time is either run with `go run` or tracked in a `tools.go` file.

Upgrading is explicit:

```
go get -u ./...          # upgrade everything (minor and patch)
go get github.com/gin-gonic/gin@v1.10.1   # one package, one version
go mod tidy
```

Major versions are in the path

This surprises everyone. From v2 onward, the major version is part of the import path:

```
import "github.com/you/myapp/v2/store"
```

It looks strange and it solves a real problem: two major versions of the same library can coexist in one build, because as far as the compiler is concerned they are different packages. You will see this in Grit itself, which imports as `github.com/MUKE-coder/grit/v3`.

init, and why to avoid it

A package can declare `func init()`, which runs automatically before `main`.

```
func init() {
    // runs once, before main, in no order you control across packages
}
```

Use it rarely. It makes startup order implicit, it cannot return an error, and it runs even when a test only wanted one function from the package. Explicit setup called from `main` is almost always better.

PROJECT 8: SPLIT THE INVENTORY INTO PACKAGES

Take the inventory from chapter seven and give it a real project layout. The point is to feel the import rules rather than to add features.

```
inventory/
├── go.mod                  module github.com/you/inventory
├── main.go                 package main
└── internal/
    └── inventory/
        ├── item.go         package inventory
        └── store.go        package inventory
```

`internal/inventory/item.go`:

```
package inventory

import "errors"

var ErrInsufficientStock = errors.New("insufficient stock")

type Item struct {
    SKU   string
    Name  string
    Stock int
```

```
}

// low is unexported, so only this package can call it. item.go and store.go
// are the same package, so store.go can use it with no import.
func (i *Item) low(threshold int) bool {
    return i.Stock < threshold
}
```

`internal/inventory/store.go`:

```
package inventory

import "fmt"

type Store struct {
    items map[string]*Item
}

func New() *Store {
    return &Store{items: map[string]*Item{}}
}

func (s *Store) Add(sku, name string, stock int) {
    s.items[sku] = &Item{SKU: sku, Name: name, Stock: stock}
}

func (s *Store) Take(sku string, n int) error {
    it, ok := s.items[sku]
    if !ok {
        return fmt.Errorf("taking %s: not found", sku)
    }
    if it.Stock < n {
        return fmt.Errorf("taking %d of %s: %w", n, sku,
ErrInsufficientStock)
    }
    it.Stock -= n
    return nil
}

// Low uses the unexported method from the other file. No import needed.
func (s *Store) Low(threshold int) []*Item {
    var out []*Item
    for _, it := range s.items {
        if it.low(threshold) {
            out = append(out, it)
        }
    }
```

```
    return out
}
```

`main.go`:

```
package main

import (
    "errors"
    "fmt"

    "github.com/you/inventory/internal/inventory"
)

func main() {
    inv := inventory.New()
    inv.Add("A1", "Blue widget", 10)

    if err := inv.Take("A1", 99); err != nil {
        fmt.Println(errors.Is(err, inventory.ErrInsufficientStock))  // true
    }

    for _, it := range inv.Low(20) {
        fmt.Println("low:", it.Name, it.Stock)
    }
}
```

Now run three experiments. Call `inv.low(5)` from `main.go` and read the error: lowercase means package-private, and the compiler enforces it. Move the whole `internal` directory to `pkg` and confirm it still builds, then put it back. And try `import "./internal/inventory"` to see that relative imports genuinely do not exist.

YOUR TURN

Add a second package, `internal/report`, with a function that takes a `*inventory.Store` and prints a summary. You will hit a design question immediately: `report` needs to read every item, but `Store.items` is unexported. Solve it by adding an exported method to `inventory` rather than by exporting the field, and notice that the package boundary just forced you into a better API than you would have written in one file.

0

Interfaces, and why Go's point the other way

If you take one idea from this book, take this one. Go's interfaces look like TypeScript's and work backwards from them, and once that clicks a great deal of Go code stops looking odd and starts looking inevitable.

The mechanical part

An interface is a set of method signatures.

```
type Shape interface {
    Area() float64
    Perimeter() float64
}
```

Any type with those two methods **is** a `Shape`. You do not say so anywhere.

```
type Rect struct{ W, H float64 }

func (r Rect) Area() float64      { return r.W * r.H }
func (r Rect) Perimeter() float64 { return 2 * (r.W + r.H) }

// Rect is now a Shape. Nothing was declared. Nothing imports Shape.
var s Shape = Rect{W: 3, H: 4}
```

This is **structural typing**, and it is called implicit interface satisfaction.

> **IF YOU KNOW JAVASCRIPT**
> TypeScript interfaces are structural too, so this much is familiar: an object with the right shape satisfies the interface. The difference is where the interface **lives** and who declares it, which is the next section and is the part that actually matters.

The direction: interfaces belong to the consumer

In Java or C#, the implementer declares what it implements: `class Rect implements Shape`. The interface is defined near the thing that provides it, and the provider must import it.

In Go, the convention is the reverse. **The interface is declared by the package that consumes it, and lists only what that consumer needs.**

Consider a function that writes a report somewhere.

```
// In package report. It does not import os, or a logging library, or a
// storage SDK. It says what it needs and nothing more.
type Writer interface {
    Write(p []byte) (int, error)
}

func Generate(w Writer, data []Row) error {
    for _, r := range data {
        if _, err := fmt.Fprintf(w, "%s,%d\n", r.Name, r.Count); err != nil
{
            return err
        }
    }
    return nil
}
```

Now `Generate` accepts a file, a network connection, an HTTP response, an in-memory buffer, a gzip stream, or anything else with a `Write` method. None of those types know `report` exists. The dependency points **from** the consumer **to** nothing at all.

> **IF YOU KNOW JAVASCRIPT**
> In TypeScript you would often import a type from the module that provides it, or define a shared `types.ts` that both sides import. Go's version removes the shared file: the consumer owns a small interface, the provider owns a concrete type, and neither imports the other. That is the decoupling, and it is why Go codebases have surprisingly few imports between their own packages.

Keep them small

That `Writer` above is the real one. `io.Writer` in the standard library is exactly those four words, and it is arguably the most successful interface ever designed:

```
type Writer interface {
    Write(p []byte) (int, error)
}
```

One method. Hundreds of types satisfy it. Everything composes.

The proverb is **the bigger the interface, the weaker the abstraction**. A one-method interface is satisfied by nearly anything and is easy to fake in a test. A ten-method interface is satisfied by one type and is a class declaration wearing a disguise.

WATCH OUT

The beginner mistake, carried in from other languages, is to define an interface for every struct up front:

```
// Do not do this.
type UserService interface {
    Create(...) error
    Update(...) error
    Delete(...) error
    List(...) ([]User, error)
    // ... nine more
}

type userService struct{}   // the only implementation, ever
```

One implementation and one interface that mirrors it exactly is pure ceremony: you have doubled the code and decoupled nothing. The Go advice is **accept interfaces, return structs**: return the concrete type from your constructor, and let whichever caller needs an abstraction declare the two methods it actually uses.

The empty interface, and `any`

An interface with no methods is satisfied by everything.

```
var x any = 42          // any is an alias for interface{}
x = "hello"
x = User{}
```

`any` is Go's `unknown`, not its `any`. You cannot do anything with the value until you establish what it is. The tool is a **type assertion**:

```
s, ok := x.(string)
if !ok {
    // x was not a string
}
```

Or a type switch, which is the readable form when several types are possible:

```
switch v := x.(type) {
case string:
    fmt.Println("string of length", len(v))
case int:
    fmt.Println("int doubled:", v*2)
case User:
    fmt.Println("user named", v.Name)
default:
    fmt.Println("something else")
}
```

> **WATCH OUT**
> The single-value assertion `s := x.(string)` **panics** if the type is wrong. The two-value form `s, ok := x.(string)` never panics. Use the two-value form unless you genuinely want to crash, which in a server you almost never do.

Reach for `any` rarely. It throws away everything the type system knows, and most uses of it in a new codebase are better served by an interface with one method or, since Go 1.18, by generics.

The nil interface trap

This one catches experienced people and is worth the page.

An interface value has two parts internally: a **type** and a **value**. It is `nil` only when both are nil.

```
type MyError struct{}
func (e *MyError) Error() string { return "boom" }

func doWork() error {
    var e *MyError = nil    // a nil pointer
    return e                // wrapped in an error interface
}

err := doWork()
fmt.Println(err == nil)     // false!
```

The returned interface holds type `*MyError` and value `nil`. The type is not nil, so the interface is not nil, so `err != nil` is true and your error handling runs for an error that is not there.

IF YOU KNOW JAVASCRIPT
There is no JavaScript equivalent because there is no such two-part value. The practical rule is the one from chapter four, and now you know why it exists: **declare the return type as `error` and return a literal `nil`**, never a typed nil variable.

```
func doWork() error {
    if somethingWrong {
        return &MyError{}
    }
    return nil          // literal nil, not a typed nil
}
```

Interfaces you will meet constantly

Four from the standard library carry an enormous amount of Go code.

`io.Reader`	`Read(p []byte) (int, error)`. Anything you can read bytes from
`io.Writer`	`Write(p []byte) (int, error)`. Anything you can write bytes to
`error`	`Error() string`. You met it in chapter four
`fmt.Stringer`	`String() string`. How a type prints itself

`Stringer` is a quick win. Implement it and `fmt.Println` uses it:

```
func (u User) String() string {
    return fmt.Sprintf("%s <%s>", u.Name, u.Email)
}

fmt.Println(u)      // Ada <ada@example.com>
```

IF YOU KNOW JAVASCRIPT
That is `toString()`, with one important difference: nothing is called implicitly during string concatenation, because Go has no implicit conversion. `"x" + u` does not compile. Only the `fmt` family looks for `String()`.

Why this matters for testing

The payoff arrives when you test. A function that takes an interface can be handed a fake with no mocking library and no dependency injection framework.

```
// In the package that needs it, listing only what it uses.
type Mailer interface {
```

```
    Send(to, subject, body string) error
}

func Register(u User, m Mailer) error {
    // ...
    return m.Send(u.Email, "Welcome", "Hello "+u.Name)
}
```

The test:

```
type fakeMailer struct {
    sent []string
}

func (f *fakeMailer) Send(to, subject, body string) error {
    f.sent = append(f.sent, to)
    return nil
}

func TestRegisterSendsWelcome(t *testing.T) {
    m := &fakeMailer{}
    if err := Register(User{Email: "ada@example.com"}, m); err != nil {
        t.Fatal(err)
    }
    if len(m.sent) != 1 || m.sent[0] != "ada@example.com" {
        t.Errorf("got %v, want one mail to ada@example.com", m.sent)
    }
}
```

Six lines of fake, no library, no configuration. That is the whole argument for consumer-declared interfaces, and it only works because `Mailer` has one method rather than the fifteen a real mail client has.

PROJECT 9: A NOTIFIER WITH THREE BACKENDS

Build something that uses an interface the way Go intends, then prove the decoupling by adding a backend without touching the code that uses it.

```
package main

import (
    "fmt"
    "io"
    "os"
    "strings"
```

```
)

// Notifier is declared here, by the consumer, and has one method.
type Notifier interface {
    Notify(message string) error
}

// Three unrelated implementations. None of them mention Notifier.

type ConsoleNotifier struct{}

func (ConsoleNotifier) Notify(msg string) error {
    _, err := fmt.Println("[console]", msg)
    return err
}

type WriterNotifier struct{ W io.Writer }

func (w WriterNotifier) Notify(msg string) error {
    _, err := fmt.Fprintf(w.W, "[file] %s\n", msg)
    return err
}

type CollectingNotifier struct{ Messages []string }

func (c *CollectingNotifier) Notify(msg string) error {
    c.Messages = append(c.Messages, msg)
    return nil
}

// The consumer. It never learns which backend it has.
func alertAll(ns []Notifier, msg string) error {
    for _, n := range ns {
        if err := n.Notify(msg); err != nil {
            return fmt.Errorf("notifying: %w", err)
        }
    }
    return nil
}

func main() {
    var buf strings.Builder
    collector := &CollectingNotifier{}

    notifiers := []Notifier{
        ConsoleNotifier{},
        WriterNotifier{W: &buf},
```

```
        WriterNotifier{W: os.Stdout},
        collector,
    }

    if err := alertAll(notifiers, "disk almost full"); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }

    fmt.Print("buffer contains: ", buf.String())
    fmt.Println("collector saw:", collector.Messages)
}
```

Notice three things. `WriterNotifier` is constructed twice with completely different destinations, a string builder and standard output, because both satisfy `io.Writer`. `CollectingNotifier` uses a pointer receiver because it mutates, so the slice holds `collector` rather than `CollectingNotifier{}`, and putting the value in instead is a compile error worth seeing. And `alertAll` would work unchanged against an email sender, a Slack client or a test fake.

Now add a fourth: a `PrefixNotifier` that wraps another `Notifier` and adds a prefix before delegating. That is the decorator pattern, in Go, in eight lines, with no base class:

```
type PrefixNotifier struct {
    Prefix string
    Inner  Notifier
}

func (p PrefixNotifier) Notify(msg string) error {
    return p.Inner.Notify(p.Prefix + msg)
}
```

YOUR TURN

Give `CollectingNotifier` a `String() string` method so it satisfies `fmt.Stringer`, then print the collector directly with `fmt.Println(collector)` and watch the output change. Then write a test for `alertAll` that proves it stops and returns a wrapped error when one notifier fails, using a fake that always errors. That test is four lines and needs no library, which is the point of the chapter.

0

Designing with errors: what a package should return

Chapter four covered the mechanics: `if err != nil`, wrapping with `%w`, `errors.Is` and `errors.As`. This chapter is about the decision those mechanics leave you with, which is harder and matters more: **when someone calls your function and it fails, what should they be able to do about it?**

Get this right and the caller writes three lines. Get it wrong and they end up matching on error message strings, which is the clearest sign of an API that did not think about failure.

Three kinds of error, and when to use each

Opaque. The caller can log it and give up. Nothing else.

```
return fmt.Errorf("parsing config: %w", err)
```

This is the default and it is correct far more often than people expect. If there is nothing useful a caller could do differently, do not invent a way for them to try.

Sentinel. A specific, exported value the caller compares against.

```
var ErrNotFound = errors.New("not found")

func (s *Store) Get(id int) (*User, error) {
    ...
    return nil, ErrNotFound
}
```

The caller:

```
u, err := store.Get(id)
if errors.Is(err, ErrNotFound) {
    http.Error(w, "no such user", http.StatusNotFound)
    return
}
```

Use a sentinel when the condition is a **fact with no detail**: not found, already exists, timed out. The name is the entire payload.

Typed. A struct implementing `error`, carrying fields.

```
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Reason)
}
```

The caller:

```
var ve *ValidationError
if errors.As(err, &ve) {
    respond(w, 422, map[string]string{ve.Field: ve.Reason})
}
```

Use a typed error when the caller needs **data** from the failure, not just its identity. A validation failure needs the field name. A rate limit needs the retry-after. A conflict needs the version that is current.

Opaque	Caller logs and gives up. The default
Sentinel	Caller branches on identity. `errors.Is`
Typed	Caller needs fields out of it. `errors.As`

WATCH OUT

A sentinel or an exported error type is **part of your public API**. Renaming `ErrNotFound`, or changing which conditions return it, breaks callers exactly like changing a function signature. Declare them deliberately and few.

The corollary: do not export an error type just because you made one. If nothing outside the package uses `errors.As` on it, keep it unexported and return it as a plain `error`.

Wrap, but not at every level

`%w` preserves the chain, and the temptation is to wrap at every function boundary. Do not. The result is a message like:

```
handling request: getting user: loading user: querying users: executing
query:
scanning row: sql: no rows in result set
```

Every layer added a word and none added information.

The rule worth following: **wrap when you add context the caller does not already have**. The database layer knows the query. The service layer knows the user ID. The handler knows the request. Each of those is worth one wrap. "I am a function and I called another function" is not context.

```
// Good: adds the thing the caller cannot see.
return fmt.Errorf("loading user %d: %w", id, err)

// Noise: the caller already knows they called LoadUser.
return fmt.Errorf("LoadUser: %w", err)
```

> **IF YOU KNOW JAVASCRIPT**
> This is the same discipline as a JavaScript stack trace, with one difference: Go errors have no stack trace by default, so the chain of wraps **is** your trace. That is why each one should earn its place. Some teams add stack traces with a library; the standard library deliberately does not, on the grounds that a well-wrapped error is more readable than sixty frames.

Error strings are a contract, lowercase and unpunctuated

Convention, followed throughout the standard library:

```
errors.New("connection refused")     // good
errors.New("Connection refused.")    // wrong: capital and full stop
```

The reason is composition. Errors get wrapped into sentences, and a capital in the middle reads badly:

```
loading config: Connection refused.: retrying
```

Also: do not end with a newline, and do not include the word "error". The reader knows.

Do not return both a value and an error

Pick one. A function that returns a partially filled struct **and** an error invites the caller to use the broken value.

```
// Bad: what is the caller supposed to do with a half-built User?
func Load(id int) (User, error)

// Good: nil on failure, impossible to misuse.
func Load(id int) (*User, error)
```

The exceptions are real but narrow: `io.Reader` returns a byte count with an error because a partial read is meaningful, and `strconv.Atoi` returns zero because it has nothing better to offer. If you are not implementing something in that shape, return nil.

Handle an error once

The most common error-handling bug in any language is doing two things with one failure:

```
if err != nil {
    log.Printf("saving user: %v", err)   // handled it here
    return err                           // and again, up the stack
}
```

Now it is logged at every level, and the operator reading the logs sees the same failure five times and has to work out whether it happened once or five times.

Choose: **either** handle it, **or** return it. The usual split is that libraries and services return, and the top layer, a handler or `main`, logs once and decides what the user sees.

Mapping errors at the boundary

This is where the three kinds pay off. One place in your program turns errors into HTTP responses:

```
func writeError(w http.ResponseWriter, err error) {
    var ve *ValidationError
    switch {
    case errors.Is(err, ErrNotFound):
        respond(w, 404, "not found")
    case errors.Is(err, ErrConflict):
        respond(w, 409, "already exists")
    case errors.As(err, &ve):
        respond(w, 422, map[string]string{ve.Field: ve.Reason})
    default:
        log.Printf("unhandled: %v", err)   // the only place that logs
        respond(w, 500, "internal error")
    }
}
```

Everything below this function returns errors and logs nothing. Everything above it gets a correct status code. That is the whole architecture of error handling in a Go service, and you will meet it again in part four, because it is exactly what Grit generates.

> **IF YOU KNOW JAVASCRIPT**
> In an Express app this is the error middleware, `app.use((err, req, res, next) => ...)`, and the design problem is identical: the middleware has to work out what kind of failure it received. JavaScript usually does it with `err instanceof ValidationError` or a `code` property. Go's `errors.As` is the same idea with the chain-walking built in, so it still works after the error has been wrapped three times.

panic is not an error

Go has `panic`, and it is not for expected failures. A panic unwinds the stack and, uncaught, kills the program.

Use `panic` for **programmer error that cannot be recovered from**: an impossible state, a nil dependency at startup, a configuration that cannot be parsed before the server starts serving.

```
func MustCompile(pattern string) *Regexp {
    re, err := Compile(pattern)
    if err != nil {
        panic("regexp: Compile(" + pattern + "): " + err.Error())
    }
    return re
}
```

The `Must` prefix is the convention: it means "this panics instead of returning an error, and you should only call it where failure means the program is broken", typically at package initialisation with a literal you wrote yourself.

Never panic for a bad request, a missing row, or a failed network call. Those are expected, and expected failures are values.

> **PROJECT 10: A USER STORE WITH A DESIGNED ERROR SURFACE**
> Build a small store that uses all three error kinds deliberately, plus the boundary mapping. This is a miniature of the structure every Go service ends up with.
>
> ```
> package main
>
> import (
> ```

```
    "errors"
    "fmt"
    "strings"
)

// Sentinels: facts with no detail. Part of the public API.
var (
    ErrNotFound = errors.New("user not found")
    ErrConflict = errors.New("email already registered")
)

// Typed: the caller needs the field name out of it.
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("%s: %s", e.Field, e.Reason)
}

type User struct {
    ID    int
    Name  string
    Email string
}

type Store struct {
    users  map[int]*User
    nextID int
}

func NewStore() *Store {
    return &Store{users: map[int]*User{}, nextID: 1}
}

func (s *Store) Create(name, email string) (*User, error) {
    if strings.TrimSpace(name) == "" {
        return nil, &ValidationError{Field: "name", Reason: "must not be
empty"}
    }
    if !strings.Contains(email, "@") {
        return nil, &ValidationError{Field: "email", Reason: "must be an
email address"}
    }
    for _, u := range s.users {
        if u.Email == email {
```

```
            // Wrapped, so the caller gets the identity and the detail.
            return nil, fmt.Errorf("creating user %s: %w", email,
ErrConflict)
        }
    }
    u := &User{ID: s.nextID, Name: name, Email: email}
    s.users[u.ID] = u
    s.nextID++
    return u, nil
}

func (s *Store) Get(id int) (*User, error) {
    u, ok := s.users[id]
    if !ok {
        return nil, fmt.Errorf("getting user %d: %w", id, ErrNotFound)
    }
    return u, nil
}

// The boundary. The only place that decides what a failure means, and the
// only place that logs.
func describe(err error) string {
    var ve *ValidationError
    switch {
    case err == nil:
        return "200 ok"
    case errors.Is(err, ErrNotFound):
        return "404 not found"
    case errors.Is(err, ErrConflict):
        return "409 conflict"
    case errors.As(err, &ve):
        return fmt.Sprintf("422 invalid %s (%s)", ve.Field, ve.Reason)
    default:
        return "500 internal: " + err.Error()
    }
}

func main() {
    s := NewStore()

    _, err := s.Create("Ada", "ada@example.com")
    fmt.Println(describe(err))                  // 200 ok

    _, err = s.Create("", "x@example.com")
    fmt.Println(describe(err))                  // 422 invalid name

    _, err = s.Create("Grace", "not-an-email")
```

```
    fmt.Println(describe(err))                  // 422 invalid email

    _, err = s.Create("Someone", "ada@example.com")
    fmt.Println(describe(err))                  // 409 conflict

    _, err = s.Get(99)
    fmt.Println(describe(err))                  // 404 not found
}
```

Two things to prove to yourself. First, `errors.Is(err, ErrConflict)` returns true even though the error was wrapped with extra text, which is the entire reason to use `%w` rather than `%v`. Change that one verb to `%v` and watch the conflict case fall through to 500.

Second, note that `Create` never logs. It returns. `describe` is the only function that decides what a failure means to the outside world, so moving this store behind an HTTP API means changing one function.

YOUR TURN

Add an `Update(id int, name string) error` that can fail in three ways: the user is missing, the name is invalid, or the name is unchanged. Decide for each whether it is opaque, sentinel or typed, and be able to defend the choice. “Unchanged” is the interesting one: is that an error at all, or should `Update` quietly succeed? There is a defensible answer either way, and the act of choosing is the skill this chapter is teaching.

0

defer, panic and recover

Three keywords that look like `finally`, `throw` and `catch` and are not. The resemblance is close enough to mislead, which is why they get a chapter.

defer: cleanup that cannot be forgotten

`defer` schedules a function call to run when the **surrounding function** returns, by any path.

```
func readConfig(path string) error {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()     // runs on every return below, and on a panic

    // twenty lines with six return statements
    return nil
}
```

Write the `defer` immediately after the thing that needs cleaning up. That adjacency is the point: open and close are one line apart, so a reviewer can see the pair without scrolling.

> **IF YOU KNOW JAVASCRIPT**
> `try { ... } finally { f.close() }` does the same job by wrapping, which indents the body and separates the close from the open. `defer` keeps them together and costs no indentation. It also composes: ten resources means ten `defer` lines, not ten nested blocks.

The three rules of defer

One: arguments are evaluated immediately, the call happens later.

```
func main() {
    msg := "first"
```

```
    defer fmt.Println(msg)   // prints "first", not "second"
    msg = "second"
}
```

`msg` was read when the `defer` statement ran. To defer the **evaluation**, wrap it in a closure:

```
defer func() { fmt.Println(msg) }()   // prints "second"
```

Two: deferred calls run last in, first out.

```
for i := 0; i < 3; i++ {
    defer fmt.Println(i)
}
// prints 2, then 1, then 0
```

This is right for cleanup: resources are released in the reverse of the order they were acquired, like unwinding a stack.

Three: defer is function-scoped, not block-scoped.

> **WATCH OUT**
> This is the one that causes real bugs:
>
> ```
> func processAll(paths []string) error {
>     for _, p := range paths {
>         f, err := os.Open(p)
>         if err != nil {
>             return err
>         }
>         defer f.Close()     // does NOT run at the end of this iteration
>         // ... use f
>     }
>     return nil
> }
> ```
>
> All the files stay open until `processAll` returns. With a thousand paths you run out of file descriptors.
>
> The fix is to give each iteration its own function, so there is a return for the `defer` to fire on:
>
> ```
> for _, p := range paths {
>     if err := processOne(p); err != nil {
>         return err
>     }
> ```

```
}

func processOne(p string) error {
    f, err := os.Open(p)
    if err != nil {
        return err
    }
    defer f.Close()     // runs when processOne returns
    // ... use f
    return nil
}
```

Extracting the loop body into a function is the standard answer, and it usually improves the code for other reasons too.

defer can change the return value

A deferred closure can modify a **named** return value after the `return` statement has chosen it. This is the one legitimate use of named returns.

```
func doWork() (err error) {
    defer func() {
        if err != nil {
            err = fmt.Errorf("doWork: %w", err)
        }
    }()

    return doSomethingThatFails()    // gets wrapped on the way out
}
```

Used sparingly this is tidy. Used often it makes a function's return value hard to trace, so prefer explicit wrapping at each return unless you have many of them.

defer and errors from Close

A deferred `Close` discards its error, which is usually fine for reading and sometimes wrong for writing: a failed `Close` on a file you wrote may mean the data never reached the disk.

```
func writeReport(path string, data []byte) (err error) {
    f, err := os.Create(path)
    if err != nil {
        return err
    }
```

```
    defer func() {
        if cerr := f.Close(); cerr != nil && err == nil {
            err = fmt.Errorf("closing %s: %w", path, cerr)
        }
    }()

    _, err = f.Write(data)
    return err
}
```

That is verbose and correct. For a read-only file, `defer f.Close()` is accepted practice and nobody will object.

panic: stop, this should not have happened

`panic` unwinds the stack, running deferred functions as it goes, and if it reaches the top of a goroutine the program crashes with a stack trace.

```
func main() {
    defer fmt.Println("this still runs")
    panic("something impossible")
}
```

```
this still runs
panic: something impossible

goroutine 1 [running]:
main.main()
    /tmp/main.go:7 +0x65
exit status 2
```

Go panics on its own for a handful of runtime faults: nil pointer dereference, index out of range, division by zero, sending on a closed channel, and writing to a nil map.

> **IF YOU KNOW JAVASCRIPT**
> `throw` in JavaScript is for **expected** failures: a 404, a validation problem, a parse error. `panic` is not. Go's expected failures are returned values, and `panic` is reserved for "this program's assumptions are broken".
>
> If you find yourself panicking in a request handler because a user sent bad input, that is the JavaScript habit talking. Return an error.

recover: only for a boundary

`recover` stops a panic, and it only works **inside a deferred function**.

```
func safe() (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
    }()

    panic("boom")
}

err := safe()
fmt.Println(err)    // recovered: boom
```

There are exactly two good reasons to use it.

One: a server that must not die because one request was bad. A panic in one handler should return 500 for that request, not take down the process serving everybody else.

```
func Recover(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                log.Printf("panic on %s %s: %v\n%s",
                    r.Method, r.URL.Path, rec, debug.Stack())
                http.Error(w, "internal server error",
                    http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}
```

Log the stack. A recovered panic with no stack trace is a bug you will never find. Every Go web framework ships this middleware, Gin included, and Grit wires it in for you.

Two: a library turning an internal panic into an error at its edge. Some parsers panic internally to unwind out of deep recursion, then recover at the public function and return an error. The caller never sees a panic. This is legitimate and rare.

> **WATCH OUT**
> `recover` only catches panics in **its own goroutine**. A panic in a goroutine you started is not caught by a recover in the function that started it:

```go
func main() {
    defer func() { recover() }()     // does not help
    go func() {
        panic("in a goroutine")      // kills the whole program
    }()
    time.Sleep(time.Second)
}
```

Every goroutine that could panic needs its own recover. This is the single most common way a Go server dies in production: a background worker with no recover, panicking on a nil map six hours after deployment.

PROJECT 11: A WORKER THAT SURVIVES BAD INPUT

Build a small job runner where one bad job must not stop the rest, and where cleanup is guaranteed. This is the panic boundary in miniature.

```go
package main

import (
    "errors"
    "fmt"
    "strings"
)

type Job struct {
    Name string
    Run  func() error
}

// runOne is the boundary. Whatever the job does, this returns an error
// rather than letting a panic escape.
func runOne(j Job) (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("job %q panicked: %v", j.Name, r)
        }
    }()
    defer fmt.Printf("  [%s] finished\n", j.Name)   // runs even on panic

    if j.Run == nil {
        return errors.New("job has no work")
    }
    return j.Run()
}
```

```
func runAll(jobs []Job) (failed int) {
    for _, j := range jobs {
        fmt.Printf("running %s\n", j.Name)
        if err := runOne(j); err != nil {
            fmt.Println("  error:", err)
            failed++
        }
    }
    return failed
}

func main() {
    jobs := []Job{
        {Name: "ok", Run: func() error { return nil }},
        {Name: "fails", Run: func() error { return errors.New("disk
full") }},
        {Name: "panics", Run: func() error {
            var m map[string]int
            m["boom"] = 1              // panic: assignment to nil map
            return nil
        }},
        {Name: "index", Run: func() error {
            s := []int{1, 2, 3}
            fmt.Println(s[10])         // panic: index out of range
            return nil
        }},
        {Name: "no work"},
        {Name: "still runs", Run: func() error {
            fmt.Println("  the runner survived four failures")
            return nil
        }},
    }

    failed := runAll(jobs)
    fmt.Printf("\n%d of %d jobs failed\n", failed, len(jobs))
    _ = strings.TrimSpace("")
}
```

Three things to notice when you run it.

The `finished` line prints for every job including the ones that panicked, because deferred calls run while the stack unwinds. That is what makes `defer` safe for cleanup in a way that a line at the end of the function is not.

The two deferred functions run in reverse order, so `finished` prints before the recover's error is assembled.

And the program exits normally with a count, rather than dying on job three. Delete the `recover` block and run it again: you get a stack trace and jobs four, five and six never run. That difference is the reason every server has this middleware.

YOUR TURN

Move the `recover` from `runOne` up into `runAll`, wrapping the whole loop, and run it again. You will catch the first panic and then the loop stops, because the panic unwound past the loop. That experiment shows exactly what "a boundary" means: it has to be inside the unit of work you want to isolate, not outside it.

0

Goroutines, against the event loop

You already write concurrent code. Every `await` you have typed was concurrency. What you have not had is **parallelism**: two pieces of your own code running at the same instant on two cores. This chapter is about the difference, because it is the whole reason Go's model exists.

The one-keyword version

Put `go` in front of a function call and it runs independently.

```
func main() {
    go fmt.Println("from a goroutine")
    fmt.Println("from main")
    time.Sleep(100 * time.Millisecond)
}
```

That is the entire syntax. No `async` on the function, no `await` at the call, no promise returned. Any function can be a goroutine; nothing about its declaration changes.

> **WATCH OUT**
> The `time.Sleep` is there because **when `main` returns, the program exits immediately**, killing every goroutine still running. Remove it and you will probably see only "from main".
>
> Sleeping is never the real answer. The real answers are `sync.WaitGroup` and channels, both below. But almost every Go tutorial starts with a sleep, and it is worth knowing that it is a placeholder for synchronisation rather than a technique.

What is actually different

In Node, your JavaScript runs on **one** thread. The event loop interleaves work: while one request waits on the database, another gets a turn. That is excellent for I/O and useless for computation, because a function that spends two hundred milliseconds doing arithmetic blocks everything.

Go runs goroutines on a pool of operating system threads, usually one per CPU core. Two goroutines can execute at literally the same moment.

	Node	Go
Unit	Promise / task	Goroutine
Threads	One for your code	One per core, by default
Waiting on I/O	Excellent	Excellent
CPU work	Blocks everything	Uses every core
Cost each	Small object	About 2 KB of stack
Practical limit	Many thousands	Hundreds of thousands

A goroutine starts with a two kilobyte stack that grows as needed. An operating system thread starts with one or two **megabytes**. That thousand-fold difference is why "start a goroutine per request" is a normal design in Go and "start a thread per request" was never viable in Java or C#.

> **IF YOU KNOW JAVASCRIPT**
> The mental shift: in Node you **structure** code so it never blocks, because blocking is catastrophic. In Go you write straight-line blocking code, and the runtime moves the goroutine off the thread while it waits. There is no colour to functions, no `async` infecting every caller up the stack. A function that reads a file looks like a function that reads a file.

Waiting properly: sync.WaitGroup

A `WaitGroup` counts outstanding work.

```
func main() {
    var wg sync.WaitGroup

    for i := 1; i <= 3; i++ {
        wg.Add(1)                    // one more to wait for
        go func() {
            defer wg.Done()          // one fewer, even if this panics
            fmt.Println("worker", i)
        }()
    }

    wg.Wait()                        // blocks until the count is zero
    fmt.Println("all done")
}
```

Three rules, and the bugs that come from breaking them:

`wg.Add` before the `go`, never inside it. Inside, `Wait` may run before the goroutine starts and see a count of zero.

`defer wg.Done()` as the first line. A panic or an early return otherwise leaves the count above zero and `Wait` blocks forever.

Pass the `WaitGroup` by pointer if it crosses a function boundary. Copying one gives the callee its own counter, and the original never reaches zero.

Since Go 1.25 there is a shorthand that removes the first two:

```
var wg sync.WaitGroup
for i := 1; i <= 3; i++ {
    wg.Go(func() {              // Add and Done handled for you
        fmt.Println("worker", i)
    })
}
wg.Wait()
```

Collecting results

Goroutines return nothing. A `go` call is a statement, not an expression, so there is no value to take.

Write into a pre-sized slice, one index per goroutine. No lock is needed because no two goroutines touch the same element.

```
type result struct {
    url string
    ms  int64
    err error
}

func fetchAll(urls []string) []result {
    results := make([]result, len(urls))      // sized up front
    var wg sync.WaitGroup

    for i, url := range urls {
        wg.Go(func() {
            start := time.Now()
            resp, err := http.Get(url)
            if err != nil {
                results[i] = result{url: url, err: err}
                return
            }
            defer resp.Body.Close()
            io.Copy(io.Discard, resp.Body)    // read it, or the connection
```

```
leaks
            results[i] = result{url: url, ms:
time.Since(start).Milliseconds()}
        })
    }

    wg.Wait()
    return results
}
```

> **IF YOU KNOW JAVASCRIPT**
> This is `Promise.all`, with the ordering preserved the same way: by index rather than by completion. The difference is that an error in one goroutine does not reject the whole batch. Here every result carries its own error, which is usually what you wanted from `Promise.allSettled` anyway.
>
> Appending to a shared slice instead of indexing would be a data race, because `append` reads and writes the slice header. Index assignment to distinct elements is safe.

Races, and the detector that finds them

Two goroutines touching the same variable, at least one of them writing, with no synchronisation, is a **data race**. The result is not "the wrong number", it is undefined.

```
func main() {
    count := 0
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Go(func() { count++ })    // RACE
    }
    wg.Wait()
    fmt.Println(count)               // 973, 981, 1000, who knows
}
```

`count++` is three operations: read, add, write. Two goroutines interleaving them lose updates.

Go ships a race detector, and it is the most valuable tool in the toolchain:

```
go run -race main.go
go test -race ./...
```

```
WARNING: DATA RACE
Write at 0x00c0000140a8 by goroutine 8:
```

```
  main.main.func1()
      /tmp/main.go:12 +0x3c
Previous write at 0x00c0000140a8 by goroutine 7:
  ...
```

WATCH OUT

Run your tests with `-race` in CI. Always. A data race can run correctly ten million times and corrupt data on the ten-million-and-first, on a different machine, under load. The detector finds it the first time the racing access happens, which turns an unreproducible production mystery into a failing test.

It costs roughly ten times the CPU and a few times the memory, so it is a CI and development tool rather than something you ship.

The fix for the counter is a mutex:

```go
var (
    mu    sync.Mutex
    count int
)

wg.Go(func() {
    mu.Lock()
    defer mu.Unlock()
    count++
})
```

Or, for exactly this shape, an atomic:

```go
var count atomic.Int64
wg.Go(func() { count.Add(1) })
fmt.Println(count.Load())
```

Bounding how many run at once

Unbounded goroutines are a real hazard. Ten thousand URLs means ten thousand simultaneous connections, which will exhaust file descriptors or get you blocked.

A buffered channel used as a semaphore caps the concurrency:

```go
func fetchAll(urls []string, limit int) []result {
    results := make([]result, len(urls))
    sem := make(chan struct{}, limit)    // room for `limit` tokens
    var wg sync.WaitGroup
```

```
    for i, url := range urls {
        wg.Go(func() {
            sem <- struct{}{}            // take a token, blocking if full
            defer func() { <-sem }()     // give it back

            // ... fetch, as before
            _ = i
            _ = url
        })
    }

    wg.Wait()
    return results
}
```

Channels are the next chapter. For now, read `sem <- struct{}{}` as "wait until there is room, then claim a slot".

> **IF YOU KNOW JAVASCRIPT**
> The JavaScript equivalent is a library like `p-limit`, because the language gives you nothing for it. Here it is two lines and a channel, which is a fair summary of the trade Go makes across the board: fewer batteries, better primitives.

When not to use a goroutine

Starting one is cheap, which makes it tempting. Three cases where it is wrong:

When the work is faster than the coordination. Spawning a goroutine to add two numbers costs more than adding them.

When you do not wait for it. A goroutine whose result nobody collects, with no `WaitGroup` and no channel, is a leak. If `main` exits it is killed silently; if it blocks forever it holds its stack forever.

When it needs ordering. Goroutines have no execution order. If step two must follow step one, write them in sequence.

> **PROJECT 12: A CONCURRENT LINK CHECKER**
> Check a list of URLs in parallel, with a concurrency limit and a timeout, and report which are broken. This is a genuinely useful tool and it exercises everything in this chapter.
>
> ```
> package main
> ```

```
import (
    "fmt"
    "io"
    "net/http"
    "os"
    "sort"
    "sync"
    "time"
)

type check struct {
    url    string
    status int
    ms     int64
    err    error
}

func checkAll(urls []string, limit int) []check {
    results := make([]check, len(urls))
    sem := make(chan struct{}, limit)
    var wg sync.WaitGroup

    client := &http.Client{Timeout: 10 * time.Second}

    for i, url := range urls {
        wg.Go(func() {
            sem <- struct{}{}
            defer func() { <-sem }()

            start := time.Now()
            resp, err := client.Get(url)
            if err != nil {
                results[i] = check{url: url, err: err}
                return
            }
            defer resp.Body.Close()
            io.Copy(io.Discard, resp.Body)

            results[i] = check{
                url:    url,
                status: resp.StatusCode,
                ms:     time.Since(start).Milliseconds(),
            }
        })
    }

    wg.Wait()
```

```
    return results
}

func main() {
    urls := os.Args[1:]
    if len(urls) == 0 {
        fmt.Fprintln(os.Stderr, "usage: check <url>...")
        os.Exit(1)
    }

    start := time.Now()
    results := checkAll(urls, 5)
    total := time.Since(start)

    // Slowest first, so the problems are at the top.
    sort.Slice(results, func(a, b int) bool {
        return results[a].ms > results[b].ms
    })

    broken := 0
    for _, r := range results {
        switch {
        case r.err != nil:
            broken++
            fmt.Printf("  ERR  %-45s %v\n", r.url, r.err)
        case r.status >= 400:
            broken++
            fmt.Printf("  %d  %-45s %dms\n", r.status, r.url, r.ms)
        default:
            fmt.Printf("  %d  %-45s %dms\n", r.status, r.url, r.ms)
        }
    }

    fmt.Printf("\nchecked %d urls in %v, %d broken\n",
        len(results), total.Round(time.Millisecond), broken)
    if broken > 0 {
        os.Exit(1)
    }
}
```

Run it against a handful of sites and notice that the total time is roughly the **slowest** request, not the sum of all of them. That is the payoff.

Three details that matter in real code. The `http.Client` has a timeout: without one, a hung server holds a goroutine forever and `wg.Wait` never returns. The response body is read and closed, because an unread body leaks the connection. And the exit code is non-zero when something is broken, which is what makes this usable in CI.

Now run it with `-race`. It will be clean, because each goroutine writes only to `results[i]`. Then introduce a race on purpose: add a shared `var count int` and `count++` inside the goroutine, and run with `-race` again to see the detector's report.

YOUR TURN

Add a `-n` flag for the concurrency limit and try the same URL list at 1, 5 and 50. At 1 the total is the sum of all requests. At 5 it drops sharply. At 50 it probably stops improving, because you are now limited by the network or by the servers themselves. Finding the point where more concurrency stops helping is the actual skill, and it is measured rather than guessed.

0

Channels, and the select statement

Goroutines run independently. Channels are how they talk. The slogan Go is famous for is **do not communicate by sharing memory; share memory by communicating**, which sounds like a fortune cookie until you have written both versions of the same program.

A channel is a typed pipe

```
ch := make(chan int)    // unbuffered
ch <- 42                // send
v := <-ch               // receive
```

The arrow always points in the direction the value travels. `ch <- v` puts a value in, `<-ch` takes one out.

```
func main() {
    ch := make(chan string)

    go func() {
        ch <- "hello from the goroutine"
    }()

    msg := <-ch          // blocks until something arrives
    fmt.Println(msg)
}
```

No `time.Sleep` this time. The receive blocks until the send happens, which is synchronisation and message passing in one operation.

> **IF YOU KNOW JAVASCRIPT**
> The closest JavaScript analogy is an async generator or a stream, with one crucial difference: a channel receive **blocks the goroutine**. There is no callback, no `await`, and no promise. The code reads top to bottom, and the runtime parks the goroutine while it waits.

If you have used `postMessage` between workers, the mental model is closer: separate lines of execution passing values, except channels are typed and synchronous by default.

Unbuffered channels are a handshake

An unbuffered channel has no room. A send blocks until a receiver is ready, and a receive blocks until a sender is ready. The two goroutines meet.

```
ch := make(chan int)
ch <- 1                 // deadlock: nobody is receiving
```

```
fatal error: all goroutines are asleep - deadlock!
```

Go detects when every goroutine is blocked and stops the program rather than hanging forever. This is a feature, and it is how you will find most channel bugs.

Buffered channels have room

```
ch := make(chan int, 3)  // capacity 3
ch <- 1                  // does not block
ch <- 2
ch <- 3
ch <- 4                  // now it blocks: the buffer is full
```

A send blocks only when the buffer is full; a receive blocks only when it is empty.

WATCH OUT

Buffering is not a performance trick to sprinkle on. The capacity is a design decision: it is how many items may be in flight before the producer must slow down. That back pressure is usually what you want, because an unbounded queue in front of a slow consumer is just a memory leak with extra steps.

Choose zero when you want a handshake, and a specific number when you know how much slack the system should tolerate. “I made it 100 so it would not block” is how a queue silently grows to 100 and then blocks anyway.

Closing, and ranging

The sender closes a channel to say “no more values”.

```
func produce(ch chan<- int) {
    for i := 0; i < 5; i++ {
        ch <- i
```

```
    }
    close(ch)           // only the sender closes
}

func main() {
    ch := make(chan int)
    go produce(ch)

    for v := range ch {  // ends when the channel is closed
        fmt.Println(v)
    }
}
```

Receiving from a closed channel returns the zero value immediately. The two-value form tells you which happened:

```
v, ok := <-ch
if !ok {
    // the channel is closed and drained
}
```

WATCH OUT

Three rules that cover every close-related bug:

Only the sender closes. A receiver closing a channel causes the sender to panic on its next send.

Closing twice panics. So does sending on a closed channel.

You do not have to close. A channel that is garbage collected is fine. Close only to signal completion to a `range` or a `select`.

Note `chan<- int` in that signature: a **directional** channel type, meaning send-only. `<-chan int` is receive-only. Using them in function parameters makes the data flow obvious and lets the compiler stop a receiver from closing.

select: waiting on several channels

`select` blocks until one of its cases can proceed.

```
select {
case msg := <-ch1:
    fmt.Println("from ch1:", msg)
case msg := <-ch2:
    fmt.Println("from ch2:", msg)
case ch3 <- "value":
```

```
    fmt.Println("sent to ch3")
}
```

If several are ready it picks one at random, which prevents one busy channel starving another.

A `default` case makes it non-blocking:

```
select {
case msg := <-ch:
    fmt.Println(msg)
default:
    fmt.Println("nothing waiting")   // does not block
}
```

> **IF YOU KNOW JAVASCRIPT**
> `select` is `Promise.race`, with two differences worth noting. It can wait on sends as well as receives. And the losing cases are not "cancelled", they simply were not chosen: the other channels still hold their values for the next receive. `Promise.race` leaves the losers running and their results are discarded, which is a frequent source of leaked work in JavaScript.

Timeouts, the most common select

```
select {
case result := <-work:
    fmt.Println("finished:", result)
case <-time.After(2 * time.Second):
    fmt.Println("timed out")
}
```

`time.After` returns a channel that delivers a value after the duration. This is the idiomatic timeout, and it is three lines.

> **WATCH OUT**
> `time.After` leaks until it fires. In a tight loop, creating one per iteration piles up timers for the full duration. For a loop, create a `time.NewTimer` once and reset it, or use a context deadline, which is the next chapter.

Signalling with a done channel

A channel carrying no data, used only to signal, is a common shape:

```
func worker(done <-chan struct{}) {
    for {
        select {
        case <-done:
            fmt.Println("stopping")
            return
        default:
            // do a unit of work
        }
    }
}

func main() {
    done := make(chan struct{})
    go worker(done)

    time.Sleep(time.Second)
    close(done)          // closing broadcasts to every receiver
    time.Sleep(10 * time.Millisecond)
}
```

Closing is the broadcast mechanism: **every** goroutine blocked on `<-done` wakes up at once, because a closed channel always yields immediately. Sending a value would wake only one.

This pattern is so common that the standard library wrapped it, and that wrapper is `context`, which is chapter sixteen. Understand the done channel first and `context` becomes obvious rather than magical.

The nil channel, and why it is useful

Sending or receiving on a nil channel blocks forever. That sounds like a pure bug and it has one real use: disabling a `select` case.

```
for {
    select {
    case v, ok := <-in:
        if !ok {
            in = nil        // disable this case, keep the loop alive
            continue
        }
        process(v)
    case <-done:
        return
    }
}
```

Once `in` is nil its case can never be chosen, so the loop keeps serving `done` without spinning on a closed channel returning zeros forever.

Worker pools

The canonical channel program: a fixed number of workers pulling from a jobs channel.

```
func worker(id int, jobs <-chan int, results chan<- string, wg
*sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs {           // ends when jobs is closed
        results <- fmt.Sprintf("worker %d did job %d", id, j)
    }
}

func main() {
    jobs := make(chan int, 100)
    results := make(chan string, 100)
    var wg sync.WaitGroup

    for w := 1; w <= 3; w++ {
        wg.Add(1)
        go worker(w, jobs, results, &wg)
    }

    for j := 1; j <= 9; j++ {
        jobs <- j
    }
    close(jobs)                     // tells the workers no more are coming

    wg.Wait()                       // wait for them to finish
    close(results)                  // now safe to close

    for r := range results {
        fmt.Println(r)
    }
}
```

The ordering is the whole lesson. Close `jobs` so the workers' `range` ends. `Wait` for the workers. **Then** close `results`, because closing it earlier would panic a worker still writing.

> **IF YOU KNOW JAVASCRIPT**
> Compare this to the semaphore from chapter thirteen. Both bound concurrency. The semaphore starts a goroutine per item and limits how many run; the worker pool starts

a fixed number and feeds them. The pool is better when goroutine startup matters or when each worker holds an expensive resource, like a database connection. The semaphore is simpler and usually enough.

PROJECT 13: A PIPELINE WITH CANCELLATION

Build a three-stage pipeline where each stage is a goroutine connected by channels, and where the whole thing can be stopped early. This is the shape of a great deal of real Go.

```
package main

import (
    "fmt"
    "strings"
    "sync"
    "time"
)

// Stage 1: produce lines. Returns a channel rather than taking one, which
// is the convention: the producer owns and closes what it creates.
func generate(done <-chan struct{}, lines []string) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        for _, l := range lines {
            select {
            case out <- l:
            case <-done:
                return          // stop early without leaking
            }
        }
    }()
    return out
}

// Stage 2: normalise.
func normalise(done <-chan struct{}, in <-chan string) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        for l := range in {
            select {
            case out <- strings.ToLower(strings.TrimSpace(l)):
            case <-done:
                return
            }
        }
```

```
    }()
    return out
}

// Stage 3: count words, fanned out across several goroutines.
func count(done <-chan struct{}, in <-chan string, workers int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup

    for i := 0; i < workers; i++ {
        wg.Go(func() {
            for l := range in {
                time.Sleep(10 * time.Millisecond)   // pretend it is slow
                select {
                case out <- len(strings.Fields(l)):
                case <-done:
                    return
                }
            }
        })
    }

    // One goroutine whose only job is to close out once every worker is
    // finished. Without this, nothing ever closes out and main hangs.
    go func() {
        wg.Wait()
        close(out)
    }()

    return out
}

func main() {
    lines := []string{
        "  The quick brown fox  ", "jumps over", "THE LAZY DOG",
        "and then", "runs away quickly", "into the woods",
    }

    done := make(chan struct{})
    defer close(done)            // every stage stops when main returns

    start := time.Now()
    total := 0
    for n := range count(done, normalise(done, generate(done, lines)), 3) {
        total += n
    }
```

```
    fmt.Printf("%d words in %v\n", total,
time.Since(start).Round(time.Millisecond))
}
```

Run it and note the time: six lines at ten milliseconds each would be sixty milliseconds serially, and with three workers it is closer to twenty.

Then do the experiment that proves the cancellation works. Change the loop in `main` to stop after the first two results:

```
n := 0
for range count(done, normalise(done, generate(done, lines)), 3) {
    n++
    if n == 2 {
        break
    }
}
```

The `defer close(done)` fires, every stage's `select` takes the `<-done` case, and all the goroutines exit. Remove the `done` channel entirely and the producer blocks forever trying to send into a pipeline nobody is reading: a goroutine leak, which in a long-running server is a slow memory leak you will eventually have to find.

Run the whole thing with `-race` as well. It is clean, because every value moves through a channel and nothing is shared.

YOUR TURN

Add a fourth stage between normalise and count that drops any line shorter than three words, and confirm the total changes. Then count how many goroutines the program runs at its peak. If your answer is not "one per stage, plus the workers, plus the closer", re-read `count` : that little goroutine whose only job is `wg.Wait()` then `close(out)` is the piece people forget, and forgetting it is a deadlock rather than a wrong answer.

0

sync: mutexes, and when a channel is the wrong tool

The previous chapter sold channels. This one is the correction: **channels are not always the answer**, and reaching for one where a mutex belongs produces code that is slower and harder to read.

The Go team's own guidance, from the wiki: use a channel to pass ownership of data or to distribute work; use a mutex to protect shared state. The rest of this chapter is what that means in practice.

Mutex: one goroutine at a time

```
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func (c *Counter) Value() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.n
}
```

`Lock` blocks until no other goroutine holds the lock. `defer c.mu.Unlock()` on the line after is the pattern: a path that returns early without unlocking deadlocks the whole program.

> **WATCH OUT**
> Four rules, each the source of a real bug:

The zero value works. `var mu sync.Mutex` is ready. Never `mu := sync.Mutex{}` then copy it around.

Never copy a struct containing a mutex. Copying duplicates the lock, so two copies protect nothing. This is why `Inc` has a pointer receiver. `go vet` catches most of these.

Lock the reads too. A read racing with a write is still a data race, and the race detector will say so. Reading an `int` is not atomic in any sense you can rely on.

Keep the critical section small. Never do I/O, call an unknown function, or send on a channel while holding a lock. That is how a lock becomes a queue and then a deadlock.

Embedding the mutex

Common style, and it reads well:

```
type SafeMap struct {
    sync.Mutex                  // embedded: m.Lock() works directly
    items map[string]int
}

func (m *SafeMap) Set(k string, v int) {
    m.Lock()
    defer m.Unlock()
    m.items[k] = v
}
```

The cost is that `Lock` and `Unlock` become part of the type's public API, so callers can lock your struct. Keep the mutex a named field when the type is exported from a package.

RWMutex: many readers, one writer

When reads vastly outnumber writes, `sync.RWMutex` lets readers run concurrently.

```
type Cache struct {
    mu    sync.RWMutex
    items map[string]string
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock()                // read lock: others may also read
    defer c.mu.RUnlock()
    v, ok := c.items[k]
    return v, ok
}
```

```
func (c *Cache) Set(k, v string) {
    c.mu.Lock()                  // write lock: exclusive
    defer c.mu.Unlock()
    c.items[k] = v
}
```

WATCH OUT

`RWMutex` is not automatically faster. It has more bookkeeping than a plain `Mutex`, so for short critical sections with moderate contention a `Mutex` often wins. Use `RWMutex` when reads are frequent **and** the critical section is long enough to matter, and confirm with a benchmark rather than a feeling.

Also: never upgrade an `RLock` to a `Lock` while holding it. That deadlocks. Release, acquire the write lock, and re-check the condition, because things may have changed in between.

sync.Once: exactly once, no matter who asks

```
var (
    once   sync.Once
    client *http.Client
)

func Client() *http.Client {
    once.Do(func() {
        client = &http.Client{Timeout: 10 * time.Second}
    })
    return client
}
```

`Do` runs the function once, ever. Concurrent callers block until the first one finishes, so nobody sees a half-built value.

IF YOU KNOW JAVASCRIPT

This is the lazy singleton you would write in JavaScript as a module-level `let instance` with an `if (!instance)` check. That check is a race in Go, because two goroutines can both see nil. `sync.Once` is the correct version and is shorter anyway.

atomic: for single values

For a counter or a flag, a lock is heavier than necessary.

```
var requests atomic.Int64

func handle() {
    requests.Add(1)
}

func report() {
    fmt.Println(requests.Load())
}
```

The typed atomics (`atomic.Int64` , `atomic.Bool` , `atomic.Pointer[T]`) arrived in Go 1.19 and are much nicer than the old function forms. They cannot be copied, which the compiler enforces.

Use atomics only for a **single** value. The moment two values must change together, you need a mutex, because two atomic operations are not one atomic operation:

```
// WRONG: another goroutine can see total updated and count not.
total.Add(n)
count.Add(1)
```

sync.Map, and why you probably do not want it

There is a concurrent map in the standard library. It is specialised, and the documentation says so plainly: it is optimised for two cases, where a key is written once and read many times, or where goroutines work on disjoint key sets.

For everything else, a plain map with a mutex is faster and far more readable, because `sync.Map` has no type safety: everything is `any` and you assert on the way out.

```
// Usually better:
type Store struct {
    mu sync.RWMutex
    m  map[string]User
}
```

Reach for `sync.Map` after measuring, not before.

When to use which

Channel	Passing ownership of a value, distributing work, signalling an event
Mutex	Protecting state that several goroutines read and write in place
RWMutex	The same, when reads dominate and the section is long

atomic	One number or flag, nothing else
Once	Lazy initialisation
WaitGroup	Waiting for a known set of goroutines to finish

The counter from chapter thirteen makes the point. With a channel:

```
// Works, and is the wrong tool.
func counter(inc <-chan int, get chan<- int) {
    n := 0
    for {
        select {
        case v := <-inc:
            n += v
        case get <- n:
        }
    }
}
```

That is a goroutine, two channels, a `select` loop that never exits, and a lifetime you now have to manage. Against:

```
var n atomic.Int64
n.Add(1)
```

The channel version is the one that looks clever and the atomic is the one you want. **Sharing a number is not communication.**

PROJECT 14: AN IN-MEMORY CACHE WITH EXPIRY

Build something with real shared state: a cache with per-entry expiry and a background cleaner. It needs a mutex, a `Once`, an atomic and a goroutine, and the combination is typical of real service code.

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

type entry struct {
    value   string
```

```
    expires time.Time
}

type Cache struct {
    mu      sync.RWMutex
    items   map[string]entry
    hits    atomic.Int64      // counters: atomic, not under the lock
    misses  atomic.Int64
    stopper sync.Once
    done    chan struct{}
}

func NewCache(cleanEvery time.Duration) *Cache {
    c := &Cache{
        items: map[string]entry{},
        done:  make(chan struct{}),
    }
    go c.clean(cleanEvery)
    return c
}

func (c *Cache) Set(k, v string, ttl time.Duration) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.items[k] = entry{value: v, expires: time.Now().Add(ttl)}
}

func (c *Cache) Get(k string) (string, bool) {
    c.mu.RLock()
    e, ok := c.items[k]
    c.mu.RUnlock()            // released before the branch below

    if !ok || time.Now().After(e.expires) {
        c.misses.Add(1)
        return "", false
    }
    c.hits.Add(1)
    return e.value, true
}

func (c *Cache) clean(every time.Duration) {
    t := time.NewTicker(every)
    defer t.Stop()
    for {
        select {
        case <-t.C:
            now := time.Now()
```

```
            c.mu.Lock()
            for k, e := range c.items {
                if now.After(e.expires) {
                    delete(c.items, k)     // deleting while ranging is
legal
                }
            }
            c.mu.Unlock()
        case <-c.done:
            return
        }
    }
}

// Close is safe to call more than once, which a Once gives us free.
func (c *Cache) Close() {
    c.stopper.Do(func() { close(c.done) })
}

func (c *Cache) Stats() (hits, misses int64) {
    return c.hits.Load(), c.misses.Load()
}

func main() {
    c := NewCache(50 * time.Millisecond)
    defer c.Close()

    c.Set("a", "alpha", 200*time.Millisecond)
    c.Set("b", "bravo", 40*time.Millisecond)

    // Hammer it from several goroutines to prove the locking holds.
    var wg sync.WaitGroup
    for i := 0; i < 50; i++ {
        wg.Go(func() {
            c.Get("a")
            c.Set(fmt.Sprintf("k%d", i), "v", time.Second)
        })
    }
    wg.Wait()

    time.Sleep(100 * time.Millisecond)   // let b expire and be cleaned

    if _, ok := c.Get("b"); !ok {
        fmt.Println("b expired, as expected")
    }
    h, m := c.Stats()
    fmt.Printf("hits=%d misses=%d\n", h, m)
```

```
    c.Close()
    c.Close()                          // safe, thanks to Once
    fmt.Println("closed twice without panicking")
}
```

Run it with `-race`. It must be clean.

Then break it three ways, one at a time, and run with `-race` after each. Change `hits` from an `atomic.Int64` to a plain `int64` and watch the detector fire. Change `RLock` to no lock at all in `Get`, same result. Finally remove the `sync.Once` from `Close` and call it twice: the second `close(c.done)` panics, which is the "closing twice" rule from the last chapter meeting a real program.

One design note worth absorbing: `Get` releases the read lock **before** checking expiry and updating counters. The counters are atomic so they need no lock, and holding a lock while doing work you do not have to is the habit that turns a cache into a bottleneck.

YOUR TURN

Add a `Len() int` method and decide which lock it takes. Then add a maximum size, evicting the entry closest to expiry when the cache is full. That second one is harder than it looks: finding the minimum requires scanning under the write lock, which makes `Set` O(n). Think about what you would need to make it fast, and notice you are now designing an LRU, which is why real cache libraries exist.

0

Context: cancellation, deadlines, and request scope

You will see `ctx context.Context` as the first parameter of almost every function in a Go codebase, and it looks like noise until you understand what it buys. It is the done channel from chapter fourteen, standardised so that every library in the ecosystem agrees on how to be cancelled.

The problem it solves

A user opens a page, your handler starts a database query, and the user closes the tab. Without cancellation the query runs to completion, the connection stays busy, and the result is thrown away. Multiply by a thousand users on a slow page and the database falls over serving nobody.

Context is a signal that travels **down** the call stack so that work can stop.

Creating one

You rarely create a context from nothing. Two roots exist:

```
ctx := context.Background()   // the root, in main or a top-level handler
ctx := context.TODO()         // a placeholder: "a context belongs here"
```

`TODO` is for when you are threading context through an existing codebase and have not reached the caller yet. It behaves identically; it exists so that `grep TODO` finds the unfinished work.

From a root you **derive**:

```
ctx, cancel := context.WithCancel(parent)
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
ctx, cancel := context.WithDeadline(parent, someTime)
```

Each returns a child and a `cancel` function.

WATCH OUT

Always call cancel, even on the timeout versions. It is not optional:

```
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
```

Not calling it leaks the timer and the goroutine watching it until the deadline passes. `go vet` catches the obvious cases and not all of them. Writing `defer cancel()` on the very next line is the habit to build.

Using one

A context has a `Done()` channel that closes when the context is cancelled, and an `Err()` saying why.

```
func work(ctx context.Context) error {
    for i := 0; i < 1000; i++ {
        select {
        case <-ctx.Done():
            return ctx.Err()           // Canceled or DeadlineExceeded
        default:
        }
        doChunk(i)
    }
    return nil
}
```

That is the shape: check between units of work, return promptly when told.

For anything that blocks, put `ctx.Done()` in the `select` directly:

```
select {
case result := <-ch:
    return result, nil
case <-ctx.Done():
    return nil, ctx.Err()
}
```

IF YOU KNOW JAVASCRIPT

This is `AbortController` and `AbortSignal`. You create a controller, pass `signal` to `fetch`, and call `abort()`. Go's version has two differences: the convention of passing it as the **first parameter of every function** is near-universal, so the whole ecosystem supports it, and contexts form a tree where cancelling a parent cancels every descendant.

Cancellation is cooperative

Nothing is killed. A context cancellation sets a flag and closes a channel; code that does not check it keeps running.

```
func ignoresCancellation(ctx context.Context) {
    time.Sleep(10 * time.Second)    // runs the full ten seconds regardless
}
```

There are no interrupts and no thread killing in Go. If a library does not take a context, it cannot be cancelled, and that is a reason to prefer libraries that do.

The standard library already honours it

This is where it pays off, because you mostly pass a context rather than check one:

```
req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := http.DefaultClient.Do(req)   // aborts if ctx is cancelled

rows, err := db.QueryContext(ctx, "SELECT ...")   // cancels the query
```

An incoming HTTP request already carries one, cancelled automatically when the client disconnects:

```
func handler(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()                    // cancelled if the client goes
away
    rows, err := db.QueryContext(ctx, "SELECT ...")
    ...
}
```

So passing `r.Context()` into your service layer, which passes it to the database, gives you end-to-end cancellation for free. That is the single most valuable thing context does, and it requires no `select` anywhere in your code.

Timeouts compose

A derived context cannot outlive its parent, which gives you sane behaviour without arithmetic.

```
func handler(w http.ResponseWriter, r *http.Request) {
    // Overall budget for this request.
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()

    // One slow dependency gets a tighter budget.
```

```
    userCtx, cancel2 := context.WithTimeout(ctx, 500*time.Millisecond)
    defer cancel2()
    u, err := users.Get(userCtx, id)
    ...
}
```

If the request's three seconds elapse first, the inner context is cancelled too, even though its own 500ms had not expired.

Values, and why to be sparing

A context can carry request-scoped values.

```
type ctxKey string
const userKey ctxKey = "user"

ctx = context.WithValue(ctx, userKey, currentUser)

u, ok := ctx.Value(userKey).(*User)
```

Use an unexported custom key type, never a plain string, or two packages using `"user"` collide silently.

> **WATCH OUT**
> Context values are the most abused part of the package. They are untyped, invisible in a function signature, and found at runtime or not at all.
>
> The rule from the standard library documentation: use them for **request-scoped data that crosses API boundaries**, such as a request ID, an authenticated user, or a trace span. Never for optional parameters. If a function needs a value to do its job, that value is a parameter:
>
> ```
> // No.
> func Send(ctx context.Context) error    // reads the recipient from ctx
>
> // Yes.
> func Send(ctx context.Context, to string) error
> ```

The conventions

Four, and they are followed almost universally.

Context is the first parameter, named `ctx`. `func Get(ctx context.Context, id int)`.

Never store it in a struct. A context describes one call's lifetime; a struct outlives calls. The exception is `http.Request`, which has its own reasons.

Never pass nil. Use `context.TODO()` if you have nothing better.

Do not pass a context to something that outlives the request. Starting a background job with the request's context means the job is cancelled when the response is sent. Use `context.WithoutCancel(ctx)` to keep the values and drop the cancellation.

PROJECT 15: A SEARCH THAT RACES TWO SOURCES WITH A BUDGET

Query two backends at once, take whichever answers first, give up on both after a deadline, and never leak a goroutine. This is context, channels and select doing the job they exist for.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "math/rand"
    "time"
)

type Result struct {
    Source string
    Hits   int
}

// search simulates a backend that takes an unpredictable amount of time and
// respects cancellation.
func search(ctx context.Context, source string, slow time.Duration) (Result,
error) {
    select {
    case <-time.After(slow):
        return Result{Source: source, Hits: rand.Intn(100)}, nil
    case <-ctx.Done():
        // Stop promptly and say why.
        return Result{}, fmt.Errorf("%s: %w", source, ctx.Err())
    }
}

// first returns whichever source answers first, cancelling the other.
func first(ctx context.Context, budget time.Duration) (Result, error) {
    ctx, cancel := context.WithTimeout(ctx, budget)
    defer cancel()              // cancels the loser, and the timer

    type outcome struct {
```

```
        res Result
        err error
    }
    // Buffered with room for both, so the loser's send never blocks and
    // its goroutine always exits. With an unbuffered channel the loser
    // would block forever on the send: a goroutine leak.
    ch := make(chan outcome, 2)

    go func() {
        r, err := search(ctx, "index-a", 300*time.Millisecond)
        ch <- outcome{r, err}
    }()
    go func() {
        r, err := search(ctx, "index-b", 120*time.Millisecond)
        ch <- outcome{r, err}
    }()

    var lastErr error
    for i := 0; i < 2; i++ {
        select {
        case o := <-ch:
            if o.err == nil {
                return o.res, nil      // first success wins
            }
            lastErr = o.err
        case <-ctx.Done():
            return Result{}, ctx.Err()
        }
    }
    return Result{}, lastErr
}

func main() {
    // Generous budget: the fast index wins.
    r, err := first(context.Background(), time.Second)
    fmt.Printf("got %+v err=%v\n", r, err)

    // Tight budget: both are cancelled.
    r, err = first(context.Background(), 50*time.Millisecond)
    fmt.Printf("got %+v err=%v\n", r, err)
    fmt.Println("deadline exceeded:", errors.Is(err,
context.DeadlineExceeded))
}
```

Three things this program teaches.

The buffered channel is a correctness requirement, not an optimisation. When `first` returns after the winner, the loser still sends. With no buffer that send blocks forever

and the goroutine never exits. Change the `2` to a `0`, run with `-race` and then inspect goroutine counts, and you have reproduced one of the most common leaks in Go services.

The `defer cancel()` is what makes the loser stop early rather than running its full three hundred milliseconds into a void.

And `errors.Is(err, context.DeadlineExceeded)` works through the `%w` wrap in `search`, which is chapter eleven's design paying off: the caller can tell a timeout from any other failure.

YOUR TURN

Add a third source and make `first` return as soon as **two** of the three agree, rather than on the first answer. Then add a `context.WithoutCancel` branch that logs the losing results to an audit sink after the winner has been returned, and confirm that work still completes after `first` has returned. That is the pattern for "respond now, finish the bookkeeping afterwards", and getting the context wrong there is why audit records go missing under load.